

DISCOVERING NEW INSIGHTS IN MOLECULE IN BIG DATA USING MACHINE LEARNING

SIOW JIA SHENG

SIB180014 / 17167920

DEPARTMENT OF PHYSICS

FACULTY OF SCIENCE

UNIVERSITY OF MALAYA

KUALA LUMPUR

2021 / 2022

UNIVERSITY OF MALAYA
ORIGINAL LITERARY WORK DECLARATION

Name of Candidate: SLOW JIA SHENG
Matric No: SIB180014 / 17167920
Name of Degree: BACHELOR OF SCIENCE (MATERIALS SCIENCE)
Project Title: DISCOVERING NEW INSIGHTS IN MOLECULE IN BIG
DATA USING MACHINE LEARNING

I do solemnly and sincerely declare that:

- (1) I am the sole author/writer of this work;
- (2) This work is original;
- (3) Any use of any work in which copyright exists was done by way of fair dealing and for permitted purposes and any excerpt or extract from, or reference to or reproduction of any copyright work has been disclosed expressly and sufficiently and the title of the Work and its authorship have been acknowledged in this work;
- (4) I do not have any actual knowledge nor do I ought reasonably to know that the making of this work constitutes an infringement of any copyright work;
- (5) I hereby assign all and every rights in the copyright to this Work to the University of Malaya ("UM"), who henceforth shall be owner of the copyright in this Work and that any reproduction or use in any form or by any means whatsoever is prohibited without the written consent of UM having been first had and obtained;
- (6) I am fully aware that if in the course of making this work I have infringed any copyright whether intentionally or otherwise, I may be subject to legal action or any other action as may be determined by UM.

Candidate's Signature

Date:

SLOW JIA SHENG

28/1/2021

Subscribed and solemnly declared before,

Witness's Signature

Date:

Name:

Designation:

DISCOVERING NEW INSIGHTS IN MOLECULE IN BIG DATA USING MACHINE LEARNING

ABSTRACT

The use of machine learning has been rising since the improvement of computation power. In recent years, the popularity of using machine learning in the materials science has risen. The vast combinational possibility of materials makes finding hidden insights and patterns through research from lab alone challenging. Therefore, a computational method that narrow down appropriate molecular structure that can relate to material property is highly desirable. In this project, data of ground-state and excited states electronic structures of 2 million molecules based on density functional theory (DFT) and time-dependent DFT at the B3LYP/6-31G* level generated by Riken Supercomputer is used. After the generation of molecular descriptors and pre-processing of data, Random Forest method - a type of machine learning is used to determine the correlation between molecular descriptors and the electronic states of the organic molecules. Such correlation will help scientists to design better organic electronic materials that can lead to better device performance. From the research, we have obtained the R^2 of prediction for properties such as HOMO, LUMO, first excited state, oscillator strength and oscillator strength (log10) as high as 0.9, 0.94, 0.93, 0.66 and 0.67 respectively. The mean absolute error (MAE) obtained are 0.14eV, 0.12eV, 0.13eV, 0.02 and 0.38 respectively. The descriptor importance ranking for each property using Random Forest model is obtained. The correlations for target and descriptors are also analyzed. This research has shown that random forest is an effective machine learning model for big data prediction of properties of the organic molecules.

Keywords: Machine learning, organic electronics, python programming, big data, descriptors

ACKNOWLEDGEMENTS

First, I would like to express my deepest gratitude to my supervisor Associate Prof. Dr. Woon Kai Lin for the chance to let me involve in this project. It was a great experience to be involved with the cutting-edge research project. I have received much motivations form Dr. Woon. With the recommendations from Dr. Woon, I had the chance to attend courses and lectures, meet different scientists from different areas, and it was a great honour.

Next, I would like to thank the Data Intensive Computing Centre (DICC) of University Malaya, and the admins. With the HPC cluster service provided, I can manage to perform my research study because this project is computationally intensive. They have also provided detailed tutorials to guide us how to interact with the technology.

Furthermore, I would like to thank Associate Prof. Dr. Goh Boon Tong for the guidance on how to carry out final year project. Dr. Goh has prepared and arranged the process to prepare for the final year project in a detailed way, thus we are clear about the project.

TABLE OF CONTENTS

Contents

ABSTRACT.....	3
ACKNOWLEDGEMENTS.....	4
TABLE OF CONTENTS.....	5
List of Figures.....	7
List of Tables	9
List of Symbols and Abbreviations	10
CHAPTER 1: INTRODUCTION	11
1.1 Background	11
1.2 Problem statement and motivations	11
1.3 Objectives of the research	12
1.4 Scope of the research	12
Chapter 2: Literature Review	13
2.1 Introduction - Background of Research	13
2.2 Theory of Machine Learning and Random Forest	15
2.3 The Molecular Descriptors.....	17
2.4 High Performance Computing (HPC).....	18
2.5 Application of Machine Learning in Materials Science.....	19
2.6 Current Progress of Machine Learning and Application	22
2.7 Weakness and Challenges.....	24
2.8 Summary	27
Chapter 3 Methodology / Experimental Methods.....	29
3.1 Data preparation.....	29
3.2 Performing Exploratory Data Analysis (EDA)	29
3.3 Model Building	30
3.4 Tuning Machine Learning Model	31
Chapter 4: Results and Discussions	32

4.1	Introduction.....	32
4.2	Results and Discussion.....	32
4.2.1	Distribution of the values of the target properties.....	32
4.2.2	Training and testing result.....	37
4.2.3	Error deviation	43
4.2.4	Descriptors Importance	47
4.2.5	Discussion	58
4.3	Summary	61
	Chapter 5: Conclusion.....	62
5.1	Significant Findings	62
5.2	Conclusion	63
5.3	Suggestion for Future Works	63
	References.....	65
	APPENDIX.....	68
	Appendix A: Ranking of the descriptors importance.....	68
	Appendix B: Programming code	91
	Appendix C: Hyperparameter of Boruta used in the research.....	142
	Appendix D: Hyperparameter of random forest used in the research	143

List of Figures

Figure 1. Splitting on a continuous predictor variable X_i by split point c .	17
Figure 2. Splitting on a categorical predictor variable X_i , with subset $S \subset \mathbf{Si}$.	17
Figure 3. Example of functional linkage for the cause and effect of porosity in the fabrication process of superalloy.	20
Figure 4. The processing-structure-property-performance relationship.	21
Figure 5. The High-throughput virtual screening process for identification of efficient TADF emitters.	22
Figure 6. Timeline and geographic distributions of materials data infrastructures and companies	27
Figure 7. Number of materials informatics projects over time.	28
Figure 8: Distribution of the HOMO value in dataset.	34
Figure 9: Distribution of the LUMO value in dataset.	34
Figure 10: Distribution of the first excited state value in dataset.	35
Figure 11: Distribution of oscillator strength in dataset.	35
Figure 12: Distribution of the oscillator strength (log10) in dataset.	36
Figure 13: Testing result of HOMO with accuracy 0.8971	38
Figure 14: Heatmap distribution of the plot HOMO.	38
Figure 15: Testing result of LUMO with accuracy 0.9397	39
Figure 16: Heatmap distribution of the plot LUMO	39
Figure 17: Testing result of first excited state with accuracy 0.9264	40
Figure 18: Heatmap distribution of the plot first excited state.	40
Figure 19: Testing result of oscillator strength with accuracy 0.6737	41
Figure 20: Heatmap distribution of the plot oscillator strength (log scale)	41
Figure 21: Testing result of oscillator strength with accuracy 0.6606	42
Figure 22: Heatmap distribution of the plot oscillator strength	42
Figure 23: Error deviation of predicted HOMO value.	44

Figure 24: Error deviation of predicted LUMO value	44
Figure 25: Error deviation of predicted first excited state value.....	45
Figure 26: Error deviation of predicted oscillator strength (log scale) value	45
Figure 27: Error deviation of predicted oscillator strength value	46
Figure 28: Top 20 descriptor ranking with target HOMO	49
Figure 29: Top 20 descriptor ranking with target LUMO.....	51
Figure 30: Top 20 descriptor ranking with target first excited state	53
Figure 31: Top 20 descriptor ranking with target oscillator strength (log scale).....	55
Figure 32: Top 20 descriptor ranking with target oscillator strength.....	57

List of Tables

Table 1: Statistics of the properties of the molecules in the dataset.	33
Table 2: Result of the Random Forest Regression prediction.....	37
Table 3: Mean and standard deviation of the error deviation	43
Table 4: Top 5 ranking descriptor importance of HOMO and their scores	48
Table 5: Correlation scores of descriptors with target HOMO	48
Table 6: Descriptors correlation in HOMO	48
Table 7: Top 5 ranking descriptor importance of LUMO and their scores.....	50
Table 8: Correlation scores of descriptors with target LUMO	50
Table 9: Descriptors correlation in LUMO.....	51
Table 10: Top 5 ranking descriptor importance of first excited state and their scores	52
Table 11: Correlation scores of descriptors with target first excited state	52
Table 12: Descriptors correlation in first excited state	53
Table 15:Top 5 ranking descriptor importance of oscillator strength (log scale) and their scores	54
Table 16: Correlation scores of descriptors with target oscillator strength in log scale	54
Table 17: Descriptors correlation in oscillator strength of log scale	55
Table 19: Top 5 ranking descriptor importance of oscillator strength and their scores.....	56
Table 20: Correlation scores of descriptors with target oscillator strength	56
Table 21: Descriptors correlation in oscillator strength.....	57

List of Symbols and Abbreviations

EDA	:	Exploratory Data Analysis
HOMO:		Highest occupied molecular orbital
HPC	:	High Performance Computing
LUMO:		Lowest unoccupied molecular orbital
MAE	:	Mean absolute error
ML	:	Machine learning
MSE	:	Mean square error
OOB	:	Out-of-bag
RF	:	Random Forest

CHAPTER 1: INTRODUCTION

1.1 Background

We have witnessed the huge data generation due to the success of DFT theory and the advancements of computing power. Since we have obtained a huge amount of data, the next step should be extracting the knowledge from these data using novel tools. Among such tools are machine learning, which are currently leading the research into the new data-science driven paradigm. There is a need to use machine learning to understand relationship between molecular properties and molecular descriptors to improve the chemical development using big data. Applying efficient data-driven approach such as machine learning is essential to accelerate the development of functional organic materials in photovoltaic technologies. [1] The application of machine learning using data precalculated from DFT or ab initio methods can provide us a very fast estimations, by using the computational inexpensive molecular descriptors, suitable machine learning models and the suitable datasets. [2] Therefore, in this research, the molecular properties for organic molecules in big data will be determined using machine learning, to prepare for better compound development.

1.2 Problem statement and motivations

We are interested to know which descriptors that can be used to describe the properties of molecules. We also want to investigate the relationship between the chosen descriptors and target properties of the molecules. Then, we are interested to study the performance using the machine learning to study those relationships in the big data. Therefore, what are interested in what are the new insights that can be obtained from correlating molecular descriptors with molecular properties such as HOMO and LUMO level, first excited state and oscillator strength using Machine learning Algorithm.

1.3 Objectives of the research

The main objectives of this research are as follows:

1. To determine the performance of machine learning in determining organic molecule properties.
2. To find out the suitable descriptor for the organic molecules using machine learning.
3. To correlate the electronic structure of the organic molecules with descriptors.

1.4 Scope of the research

The amount of data generated is increasing vastly because of the prevalent use of computer calculation. In the view of vast chemical space that is not able to be explored by humans alone, the roles of computer in searching suitable chemicals are important. This 8 month of research was restricted to perform machine learning on organic molecules. By using computational techniques, the prediction of the result is expected to give us information about the relationship between the descriptors and target molecular properties. This research is conducted with data storage and computation will be carried out in the HPC cluster provided by Data Intensive Computing Centre (DICC) of University of Malaya.

CHAPTER 2: LITERATURE REVIEW

2.1 Introduction - Background of Research

In the past, inorganic silicon and gallium arsenide semiconductors have been the backbone of the semiconductor industry. However, the organic materials having highly conjugated system have been discovered to be able to transport charges when carriers are injected. This was discovered by Kallmann and Pope, as a current was produced when photons were radiated onto anthracene crystal with electrolyte electrodes.[3, 4]

Organic semiconductors are the materials that mostly made up by carbon and hydrogen atoms, and other heteroatoms like sulfur, oxygen and nitrogen. These materials show the properties of a semiconductor material. In organic molecules, the highest occupied molecular orbital (HOMO) is the π -orbital, while lowest unoccupied molecular orbital (LUMO) is the π^* -orbital. They are frontier orbital that governs the electrical processes in molecule. Other properties like dipole moments are the results of uneven spatial charge distribution in a molecule, singlet and triplet excited states excitons having two antiparallel and parallel electrons respectively in which one of it is at the excited state.[5] Because the materials properties can be tuned by chemically changing the molecular structure, the design of the materials can be optimized for a particular application. Due to the mechanical flexibility of the organic materials, organic electronics may provide new wearable technologies and new applications through the improvements of the materials. Although organic electronics is a new area, there is a growing commercial interest because of the performance improvements in OLEDs, organic conducting and semiconducting materials.[6]

However, the chemical space of organic materials is very vast, and it is not possible to explore by experimental lab alone. Due to its vastness, big data analysis approach might help to explore such a vast space. The term big data refers to the massive data sets having large, complex structures and have difficulties in storing, analyzing, and visualizing for further process or results. Commonly big data is useful for companies or organizations to gain richer and deeper insights from the information, hidden

patterns, and secret correlations by using big data analytics. Big data has become the center of modern science and business nowadays.[7]

However, given a large amount of data, it is not possible for human to browse through such information. Programming the computer with appropriate algorithms to browse through and analyze such data becomes important. Machine learning (ML) are computer algorithms that are capable to learn and improve the performance of a task using their own previous experience. During the past 20 years, ML has steadily become more mathematical and successful in applications. ML has the potential to amplify every aspect of working scientist's progress and allow computer systems to generate scientific thinking.[8]

For thousands of years, materials science composed only of empirical science, of metallurgical observations using stones, bronze, iron and steel, which is based on trial-and error experiments. A few centuries ago, came the paradigm of theoretical models and generalizations[9], characterized by using the various laws in the form of the mathematical equation such as the laws of thermodynamics. The second paradigm of materials is guided by theories. With the advent of computers a few decades ago, comes the third paradigm of computational science, allowing us to do simulations about complex real-world problem based on the theoretical models of the second paradigm. In the field of materials science, the examples are density functional theory (DFT) and molecular dynamics (MD) simulations. Finally, in recent years, the amount of data generated by the experiments and simulations has given rise to the fourth paradigm of science, which is big data-driven science that unifies the first three paradigms of experiment, theory, and computational simulation. [10, 11]

One of the first attempts to use materials information was the development of the Calculation of Phase Diagrams (CALPHAD) method and database in the 1970s. It performs multiple calculations of phase diagrams and puts them in the centralized database to accelerate the design and development of new alloys. In the 1990s, researchers explored the potential of data science in scientific research due to the increasing capability to collect, store and analyze "big data". In that time, Materials scientists at the Massachusetts Institute of Technology (MIT) developed tools to make predictions of materials properties from datasets, and researchers at the Technical University of Denmark demonstrated the

potential of evolutionary algorithms in finding materials with specific properties and narrow down the number of experiments needed to be done. From the envision of building a “virtual materials laboratory”, a curated database Materials Project was launched. As big data and data science are prominent, the US government announced the launch of the Materials Genome Initiative (MGI) in 2011. This initiative emphasized the application of data informatics in the field of materials discovery and design.[9] They envision the discovery, development, manufacturing and deployment of advanced materials, with twice the speed and at a fraction of cost.[10]

2.2 Theory of Machine Learning and Random Forest

Machine learning can be divided into supervised learning, unsupervised learning, semi-supervised learning as well as reinforcement learning. Traditional machine learning models like support vector machine (SVM) and decision tree (DT) are mainly used for linear classification. SVM is a generalized linear classification for the binary classification of data and had been used to classify compounds related to target drugs and similar target drugs. DT is a method for approximating discrete function values. Those are the shallow learning methods including artificial neural network (ANN) and Naïve Bayes classifier. Deep learning is an enhancement for shallow learning methods, including convolutional neural network (CNN), recurrent neural network (RNN) and others. They can be used for image and speech recognition, natural language understanding, biomedicine and other fields.[12]

We shall focus on the Random Forest method. Over the 20 years, the application of Random Forest (RF) classifier has received rising attention because of its excellent classification results and the speed of processing. It is a classification using predictions derived from an ensemble of decision trees. The RF classifier is an ensemble classifier that uses a set of CARTs (Classification and Regression Tree) to make a prediction. By drawing a subset of training samples, the trees are created. About 2/3 of the samples are used to train the trees, while the remaining are used for internal cross-validation techniques to estimate how well the resulting model performs. Each decision tree is independently produced, and each node is split by user-defined number of features (Mtry) selected at random, without pruning. Growing the forest up to a user-defined number of trees (Ntree), the algorithm will make trees that consist of high variance and low bias. Taking average class assignment probabilities calculated by all

produced trees, the final classification decision is taken. New unlabelled data input is evaluated against all decision trees in the ensemble, and each tree vote for class membership. The finally selected membership class is the one with maximum voters.[13]

To look at more details in the Random Forest, a Random Forest is a tree-based ensemble with each tree depending on a collection of random variables. It can be described as a p -dimensional random vector $X = (X_1, \dots, X_p)^T$ represents the input or predictor variables, while random variable Y represents the output, we assume an unknown joint distribution $P_{XY}(X, Y)$. The aim is finding a prediction function $f(X)$ for predicting Y , and is determined by a loss function $L(Y, f(X))$. The expected value of loss $E_{XY}(L(Y, f(X)))$ should be minimized. For choices of L in regression, the L is squared error loss $L(Y, f(X)) = (Y - f(X))^2$, while for classification, it is zero one loss:

$$L(Y, f(X)) = I(Y \neq f(X)) = \begin{cases} 0 & \text{if } Y = f(X) \\ 1 & \text{otherwise} \end{cases}$$

Minimizing the expected value of loss for squared error loss gives the conditional expectation $f(x) = E(Y|X = x)$, which is also called regression function. For the classification situation, if the set of possible values of Y is denoted by γ , minimizing $E_{XY}(L(Y, f(X)))$ for zero, gives $f(x) = \arg \max_{y \in \gamma} P(Y = y|X = x)$, which is known as Bayes rule.[14]

Ensembles construct f in terms of a collection of “base learners” $h_1(x), \dots, h_J(x)$ and these base learners combined to form the ensemble predictor $f(x)$. The base learners are averaged in regression $f(x) = \frac{1}{J} \sum_{j=1}^J h_j(x)$, while in the classification, $f(x)$ is mostly predicted class voting $f(x) = \arg \max_{y \in \gamma} \sum_{j=1}^J I(y = h_j(x))$.

The trees in Random forests are based on binary recursive partitioning trees in the predictor space using a sequence of splits on each variable. The root node includes the entire predictor space, and the terminal nodes are the end of split, which form the final partition of the predictor space. According to the value of predictor variables, the nodes are split into two left and right descendant nodes. For continuous predictor variable X_i , the split is determined by split points c , which the predictor smaller than the split point goes to the left, and the larger goes to the right.

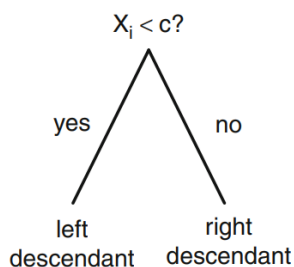


Figure 1. Splitting on a continuous predictor variable X_i by split point c . [14]

Meanwhile, the categorical predictor variable X_i takes value from a finite set of categories $S_i = \{s_{i,1}, \dots, s_{i,m}\}$. The split separates the subset of categories into left and right node.

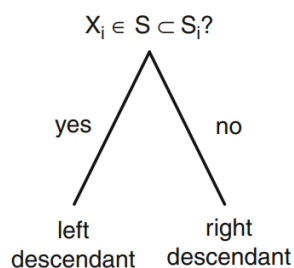


Figure 2. Splitting on a categorical predictor variable X_i , with subset $S \subset S_i$. [14]

To explain how the separation is made, it is considered by choosing the best possible split according to some criterion. For the regression, the splitting criterion is the mean squared residual at the node, while for the classification the typical splitting criterion is the Gini index. The mean squared residual, Q is represented by $Q = \frac{1}{n} \sum_{i=1}^n (y_i - \bar{y})^2$, and the $\bar{y} = \frac{1}{n} \sum_{i=1}^n y_i$ is the predicted value at the node. While the Gini index, for K classes from 1 to K , the Q is represented by $Q = \sum_{k \neq k'}^K \hat{p}_k \hat{p}_{k'}$, and $\hat{p}_k = \frac{1}{n} \sum_{i=1}^n I(y_i = k)$ which is the proportion of class k observation in the node. [14]

2.3 The Molecular Descriptors

In this project, the molecular descriptors are used. The molecular descriptors are the molecules in formally mathematical representations, which are derived by the algorithm applied to a defined molecular representation. The descriptors guide important roles in chemistry, toxicology, quality control and pharmaceutical sciences. It transforms the chemical information contained in the symbolic representation of the molecules into numbers that can be applied in the research. [15] Currently, there

are a vast number of molecular descriptors, which are simply be categorized into 1D, 2D and 3D descriptors. For example, the 1D descriptors capture bulk properties like molecular weights, while the 2D descriptors describes two dimensional representations like connectivity indices, in the meantime 3D descriptors contains conformations of molecules like solvent-accessible surface area. Some examples of descriptors are “weight” for molecular weight, “a_nN” for the number of nitrogen atoms in the molecule, “b_double” to represent the number of double non-aromatic bonds.[16]

2.4 High Performance Computing (HPC)

In this project, the HPC infrastructures will be used for the few hundreds of GBs of storage and computation. HPC is also known as supercomputer, and it is a clustered of networked compute nodes which contains processing cores and memory. HPC can solve complex computational problem using resources pooling together in the fields of research such as computational chemistry, materials science, molecular biology, and weather forecasts.[17] The trend today in the scientific problems are continuing to rely on the ability to collect, store, analyse and compute on the large data. Among the attributes of big data, volume and velocity are the challenges that imply the requirements for scalability and inevitably the use of HPC.[18] HPC is also an important part in the drug discovery process. There are millions of potential drug compounds, and computational approach has been a strategy for discovery and design such as ligand interactions and molecular dynamics. The large quantities of data can be analysed in short time, and it can discover novel molecules that remains hidden. The application of HPC is a must nowadays as the computational power to process all the chemical databases accurately and quickly is huge. For example, in specific contexts, when there are infectious disease outbreaks such as Zika virus in Brazil 2016, the HPC allows the repurpose of old drug Novobiocin in two days’ calculations, compared to the development of vaccine which takes about 3 to 6 months. The HPC based virtual screening approach, which is computational method for predicting hit molecules against target of interest, have been reinforced with the machine learning and deep learning method. In virtual screening, machine learning techniques train the models to predict a potential inhibitor that will bind to a target or structure.[19] Furthermore, as the artificial intelligence (AI) is evolving for the big data challenges, the generation rate of data and volume outstrip the computing capabilities of available

cyberinfrastructure platforms. This drives the confluence of AI and HPC to enable the data-driven discovery. Plasma physics, cosmology and materials science are among the recent accomplishments of the convergence of AI and HPC in the HPC ecosystem.[20] As machine learning is a subfield of AI, this gives us an idea to perform machine learning with our large datasets using HPC infrastructure.

2.5 Application of Machine Learning in Materials Science

So, how are the materials' big data can be applied in the fields of materials science? We shall look at the concept of PSPP (Processing-structure-properties-performance) map first. This is a visual highlight of fundamental drivers and multiscale linkages inherent in each materials system, and is an exercise to understand the problem and the performance requirements of developing new materials. The materials engineer can clearly see the key linkages that govern the materials system, and apply this knowledge in designing and optimizing the materials system. The PSPP map is also a tool to identify the pedigreed information that is important in metadata for materials data curation systems. It represents the key microsubstructural subsystems, primary links of these subsystem and properties they control, as well as the stages of processing that governs their dynamic evolution. The figure below shows the additive manufacturing of Ni-base superalloy process, with the target to determine required performance for hot section components in gas turbine engines. The highlighted components show the processing steps that cause porosity and the subsequent properties that it will affect. [21]

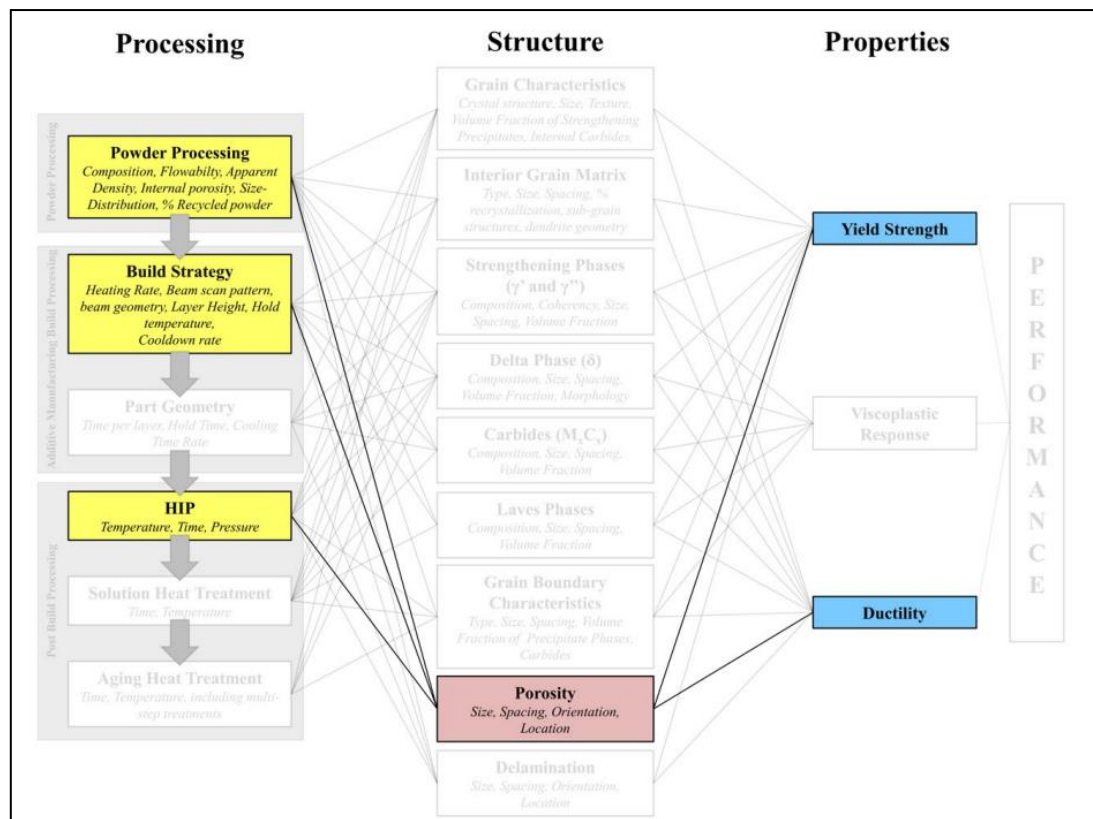


Figure 3. Example of functional linkage for the cause and effect of porosity in the fabrication process of superalloy.[21]

This is particularly useful when we want to develop the computational, data informatics and experimental tools to model the linkage in the system. This is because the addition of new linkages into the model can be viewed by highlighting it, and model accurately the structural transformations in new materials system.[21] The materials informatics and big data can be applied based on the PSPP relationships. This relationship describes that the materials informatics can either generate forward models for predictive analytics, as well as generate inverse models for optimization and design. It is shown in the diagram below.

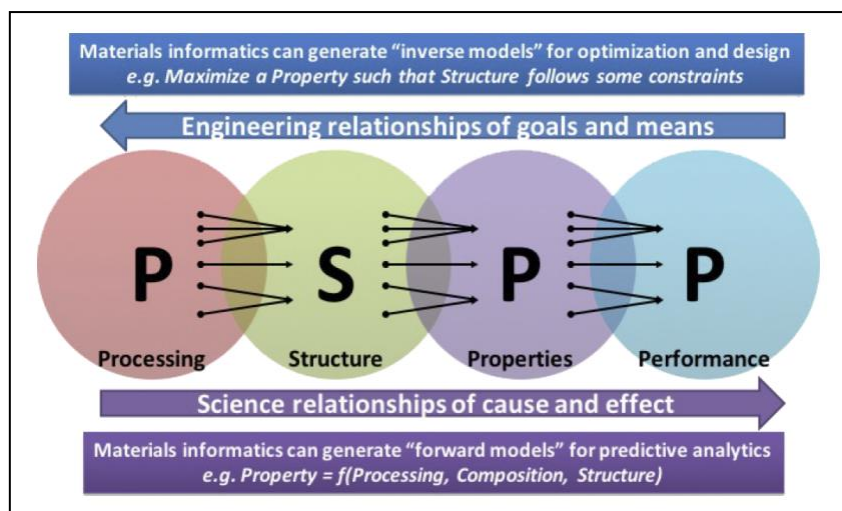


Figure 4. The processing-structure-property-performance relationship.[10]

Each experimental observation and simulation such as measurement, composition and structural parameters is a data forward model, while the database of the data can make predictive analysis to build data-driven forward models that can run in a short time compared to experiment or simulation. This acceleration can guide future simulations and experiments, and realize the much more challenging inverse models for materials discovery and design. [10]

Generally, materials informatics can only perform by those who have programming experience and data science. This is because the use of tools to perform data analysis and manipulation needs some degree of programming ability. However, many scientists without such background, and it is important that materials informatics are democratized to allow the spread to benefit of large-scale materials data analysis. To encourage this, intuitive and easy to use software must be developed for those who do not have a background in computer science or data science. [11]

In this literature review, the progress of machine learning and big data in materials science is summarized. The strengths and challenges of machine learning will be presented in this literature review. We will also see the overview and methods to perform data analysis and machine learning. In this project, the Random Forest method is used to determine the suitable descriptors in the given dataset. We will use machine learning to determine how the structure of the molecules can be described by descriptor, and how to improve the performance of organic electronics by looking at the structure affecting the HOMO and LUMO level of molecules.

2.6 Current Progress of Machine Learning and Application

The examples of using machine learning in the discovery of material properties are shown below. Thermally activated delayed fluorescence (TADF) is a family of materials that may drastically reduce the cost of phosphorescence emitters, in addition in making improvements in the operational lifetimes of devices emitting in the blue spectral range. There are thousands of potential candidates in the chemical space to synthesis for highly efficient TADF molecules. By using high throughput calculations, machine learning algorithms and experimental validation, the fast screening of more than 1.6 million molecules to identify the TADF molecules can be carried out. Below shows the commonly used high throughput screening process, as shown by this research.

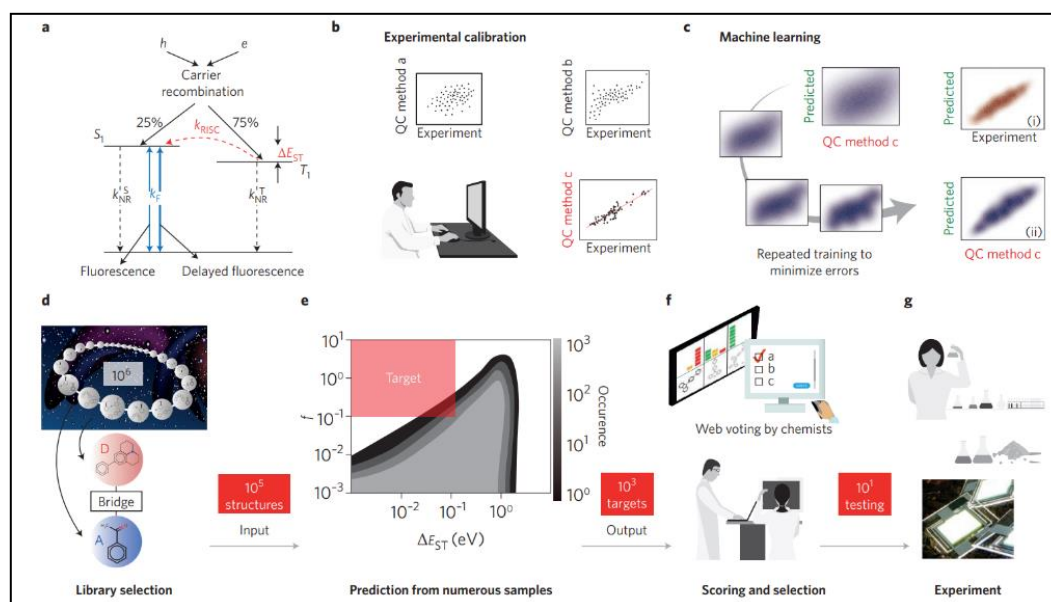


Figure 5. The High-throughput virtual screening process for identification of efficient TADF emitters.[22]

The Figure 5a shows the light-extraction process from TADF emitters. The researchers then tested several computational algorithms, to estimate the oscillator strength (f) and singlet-triplet energy difference (ΔE_{ST}) against parameters extracted experimentally from known molecules. They then compare the data from samples with different quantum calculation (QC) methods as shown in Figure 5b. The physical parameters are then predicted. In Figure 5c, the neural network by machine learning is applied to optimize the correlation between physical parameters from prediction results with the calculation from QC methods and experimental values. The trained network was then being applied to calculate the statistical distribution of the relationships between f and ΔE_{ST} as shown in Figure 5d.

This generates a library of molecular structures with potential TADF properties that consist of donor and acceptor linked by a bridge. The statistical distribution of photophysical parameters derived by the neural network is plotted in Figure 5e and the selection of promising f and ΔE_{ST} was shown to fall in the target area, which has the high electron-photon conversion efficiencies. The best TADF candidates are then selected by web voting from a pool of experts in Figure 5f. Finally, the researchers synthesis the molecules and confirmed the values matched with the prediction. The order of numbers of molecules decreased throughout each stage, from 10^5 to 10^1 molecules. This shows the effectiveness of the high-throughput virtual screening process, with approximately 2 months for a 10000 cores supercomputer to screen through 1.6 million molecules.[22]

Steel fatigue prediction can also be carried out using computational methods. The predictive models for fatigue strength of steel were carried out by Agrawal *et al.* using the data from the Japan National Institute of Materials Science (NIMS) Materials database. Extremely high cost and time of fatigue testing and potentially disastrous consequences of fatigue failures lead to the need of accurate predictions of fatigue strength. The materials informatics approach includes a series of steps that include data pre-processing, ranking -based feature selection, predictive modelling and model evaluation using leave-one -out-cross-validation. Evaluation of twelve regression-based predictive modelling methods were made, and mostly have high accuracy with a high R^2 value. This works shows that using the data science method to build models on experimental data, provide a connection of processing and composition with performance using PSPP relationships.[10]

In discovering stable compounds, a similar predictive analytics framework on simulation data using DFT calculations was carried out by Meredig, Agrawal *et al.* To perform a DFT simulation on the materials, the composition and crystal structure as input is required. Using a database of existing DFT calculations to build forward models predicting the property of material using a variety of attributes can be derived solely based on materials stoichiometric composition. The predictive models are trained on DFT data and the built model can be used to predict the formation energy of new materials without crystal structure as input. The result was found with models with high R^2 score, the forward

model could predict formation energy without any structural input faster than DFT and high accuracy.[10]

To improve the prediction of molecular descriptors for drug discovery, Gaspar Cano et al. (2016) examined a Random-Forest (RF) based approach to automatically select molecular descriptors of training data for ligands of kinases, nuclear hormone receptors and other enzymes. This reduction of features reduced the computing time and allow the exploration of much larger sets of experimental data. The two uses of RF in this study are feature ranking and dimensionality reduction, and classification using an automatically selected feature subset. In the research, they have proven the power of automatic selection of molecular descriptors using Random Forest, and avoided the manual selection of descriptors called as ad hoc. The application of Random Forest improves the accuracy of the classification methods, as well as reduces the computational cost. The results also show that the Random Forest method outperforms classification results done by SVM (support vector machine) and NN (Neural Network) method. They concluded that this method can be used to improve drug design and discovery. From their results , it is a very rational step in our project in determining the suitable descriptors using the Random Forest approach.[23]

Therefore, we can see that the application of machine learning in materials science provides strengths in their short development cycle and low costs. This can be realized with the advancement of better computation power.

2.7 Weakness and Challenges

The field of materials informatics is still in its nascent stage, there are some problems and challenges that need to be overcome. There are some challenges for the sharing of data within the field of materials science. Materials researchers today have minimal incentives to share data compared to the biological sciences and astronomy. There are also no unique sets of data sharing requirements, norms and incentives that are directly transferable to materials. Besides, the materials community lacks an obvious forcing function that necessitates a culture of structured data access and advanced analytics. Fortunately, the situation is slowly changing.[11]

A large amount of data is also the primary challenge in materials science, while the characteristics such as collection, storage, retrieval, analysis, and visualization of such data make it even more challenging. Given an example, there are data from many sources which are heterogeneous, consists of different types, with unknown dependencies, inconsistency, missing and not reliable, generation rate out of traditional system capable to handle and privacy issues.[10] These issues are summarized by the phraseology of four V's: volume, variety, veracity, and velocity. Data volume refers to the size of the data set. Data veracity is the accuracy and uncertainty of the data if they can be used repeatedly for the common good for the community. While data variety is concerned that the data take all forms in materials science ranging from discrete numerical values to qualitative descriptions of materials behaviour. Data velocity refers to the dynamics of the evolution of data, which makes us see the new information relate to the real time data acquisition.[24] However, in the materials science fields, openly accessible data are rather limited. Fortunately with the support of MGI and other efforts, the world is promoting the availability and accessibility of digital data in materials science.[10]

The materials data infrastructure includes simple databases, data centres and material discovery platforms, and it needs to give relevant data and information to be adopted by scientific communities, industries, governments, or the public. The relevance of the data is determined by the volume, type, quality, completeness, and homogeneity of data. The challenge is to develop general interdisciplinary infrastructures to be adopted by different communities for different specifications. We would also like to increase the acceptance of data infrastructures to be applied in machine learning approaches. For the data to be informative in machine learning algorithms, its features and properties need to be relevant and complete.[9]

Besides, the ideal completeness of data is difficult to obtain. Data infrastructures today facing a severe completeness problem: there are mostly computational data and no experimental data. It is important that we facilitate a seamless comparison between computational and experimental data. Efforts to further evaluate, select, and experiment need to be carried out to build synergies among computational and experimental database for data-driven materials science. [9]

There are also problems like not enough high-quality data related to materials, and the properties of materials are difficult to represent perfectly. The prediction accuracy is also lower than DFT calculations.[12] Further directions are proposed that may contribute to the solution to the problems above. First, the acceleration of the materials database is highly needed for the future development of this field. There are huge amounts of materials data from the scientific literature and experimental records such as molecular properties, reaction conditions and synthetic formulations. We can use text mining techniques to extract and collect these scattered data into the material database for machine learning. Then, the establishment of new principles for machine learning is important. The understanding of how the meaning and selected features obtained from the neural network is important to enhance the generalizability of machine learning, and identify the natural laws remain hidden to human. Next, combining the DFT theory and machine learning can increase the prediction accuracy of the model, thus providing a tool for predicting complicated properties and structures of molecules. Finally, a more accurate model based on a small accurate data set is necessary to increase the accuracy.[12]

There are also some criticisms about the machine learning algorithms in science. The machine learning is considered lack of novel laws, understanding and knowledge in their application. They are often treated as black boxes that are too complex and distinct for us to understand. The results do not indicate us the new physical laws and their theory is out of our understanding. This leads to the main concern that the occurring of models might fail in unexpected cases when we have a very good result of prediction. This may lead to catastrophic outcome where a single failure is extremely expensive and deadly. [25]

2.8 Summary

In summary, machine learning and data-driven approaches are being applied in materials science, and the higher usage the higher the acceptance of the materials infrastructure, which are databases, datacentres, and materials discovery platforms. These infrastructures grew historically from the simple databases, into data centres, and eventually moved to the materials discovery platforms.

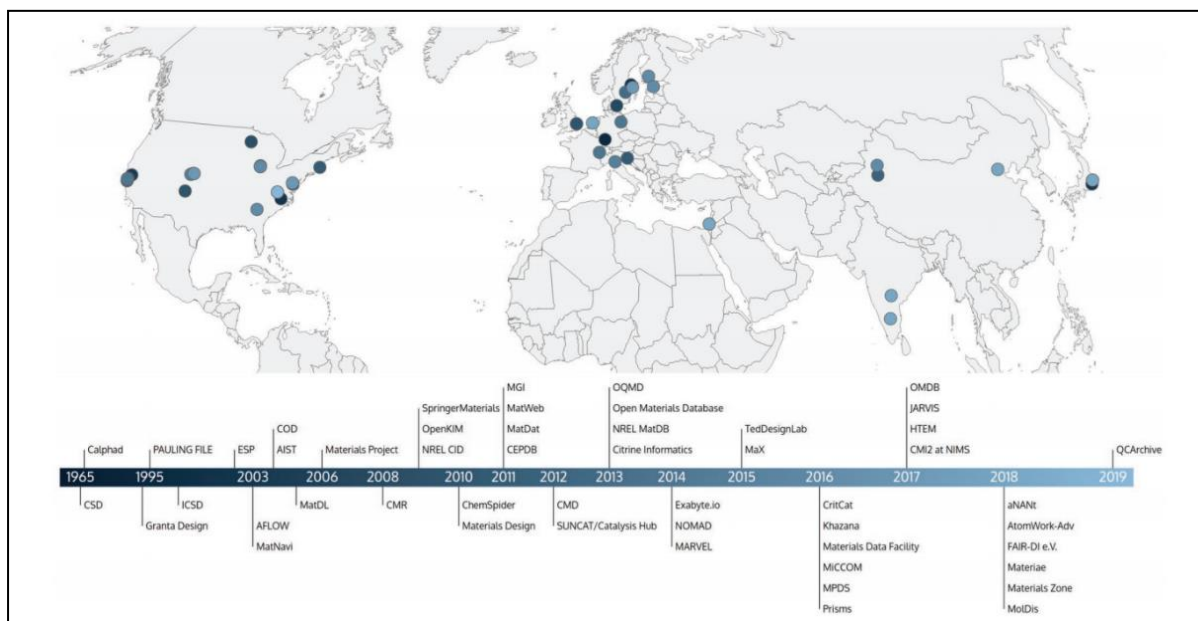


Figure 6. Timeline and geographic distributions of materials data infrastructures and companies.[9]

The Figure 3 above shows the geographic distribution of materials data infrastructures and companies database with their time of establishment as colour dots. In the past, there are more research and studies accumulate in the U.S. and Europe, but eventually, the country in Asia catches up over time. The open-access materials data are eventually available worldwide. We can also see that the number of material informatics project and infrastructures increase over time as shown below. The three divided regions are the evolution of the data infrastructure over the years.[9]

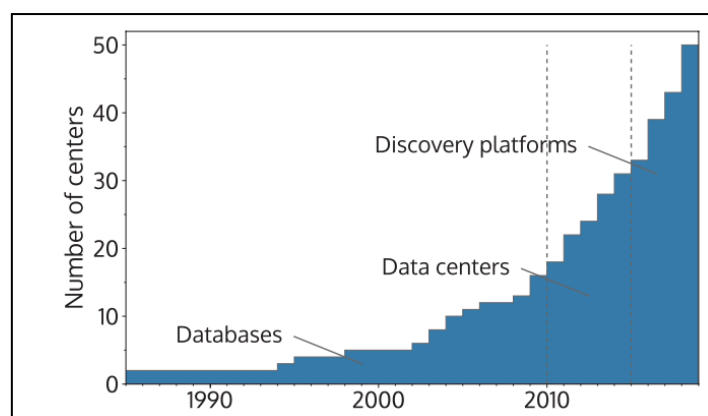


Figure 7. Number of materials informatics projects over time.[9]

The new applications of this new form of materials science will lead to the new challenge in machine learning methods, and machine learning methods will be further developed to overcome these challenges. Meanwhile, the synergy between academic developments and industrial interest is a major challenge, but this does not stop the dramatic rise in data-driven materials science in this new paradigm. Without a doubt, we have only experienced a glimpse of this data evolution.[9]

CHAPTER 3 METHODOLOGY / EXPERIMENTAL METHODS

3.1 Data preparation

First, the descriptors are generated from Simplified Molecular Input Line Entry (SMILES) using python script. The generation process took about 30 days, the number of SMILES is about 4 millions and the descriptors generated consist about 2 thousands of columns. Since the personal computer cannot handle such large project, throughout the research the storage and computation will be carried out in the HPC cluster provided by Data Intensive Computing Centre (DICC) of University of Malaya. Small samples of the project are extracted to perform testing and debugging using personal computer during each stage of research.

The molecular properties data of whole B3LYP dataset (b3lyp_2017.tar) which generated by Riken Supercomputer are downloaded and extracted using python script from <http://pubchemqc.riken.jp/>. The downloaded data is about 2 TB. The data is downloaded using rclone and stored in the HPC storage. The data are then extracted from each file using cclib python library and saved into a csv file. The extraction was performed automatically by iterating through different directories or folders. The molecular properties data that we want to extract and investigate are their HOMO, LUMO, first excited state and oscillator strength.

3.2 Performing Exploratory Data Analysis (EDA)

The descriptors are going through EDA using python library pandas. The numerical columns that contain strings and symbols is stripped to contain only numbers. The rows that contain "NA" noises are also removed. Since the datasets are big, we loaded them by chunks. The duplicated rows are also removed. The infinity values are replaced with the largest value in the column. Then, we removed the columns which have ratio between mean and median of more than 300, or remove the columns where median is equal to 0 but mean greater than 300 to avoid extreme values. This is because the input data for ML need to have a Gaussian like-distribution as the derivation of most ML theory is based on this assumption. Then, we calculate the standard deviation over mean of the columns. Those descriptors that

have standard deviation over mean less than 0.1 will be removed, since they do not give us much information because the variation is almost constant. Finally, all the data that remains are concatenated together in a large file.

The statistical analysis of HOMO and LUMO are observed. Those with rows HOMO value that are less than -8 eV, and more than -3 eV will be removed as they represented unphysical values in real world. While for the LUMO those value that are less than -4 eV and more than 0 eV will be removed, as the value above 0eV is not stable at excited state. This is because for organic molecules, they are unlikely to have stable compound outside of these ranges. Then, we also change the oscillator strength to log scale.

Finally, the two datasets descriptors (features) and molecular properties (target) are then concatenated together, as we need both the features and targets together to perform the machine learning model building. The final big dataset remaining that we have to perform machine learning have 2331362 rows of molecules and 1154 columns of descriptors.

3.3 Model Building

Before the data are feed into machine learning model, the dataset needs to be split into target and features first. The features are the descriptors, while the targets are HOMO, LUMO, first excited state and log of oscillator strength. Then, the whole dataset is divided into training and testing set. 80% of the dataset is divided into training set, while the remaining 20% of the dataset is divided into testing set. The model chosen is Random Forest Regressor, since the data values are numerical. Besides, RFR is easy to implement and produce less variance during application. The Random Forest is first performed using `n_estimators = 100` to see the performance. Since the dataset still contains thousands of columns of descriptors after EDA, the computation is costly and time consuming.

3.4 Tuning Machine Learning Model

To save computation power and time consume, the dataset is first removed highly correlated descriptors. The threshold of the correlation is set at 0.90, the descriptors that contain correlation higher than the threshold will be removed. This results about half of the descriptors are removed from the dataset. Then, the descriptors are then undergone calculation of Boruta, which is a python library, to determine whether it is important to remain or drop, to further reduce the features. The source code of Boruta can be obtained from GitHub https://github.com/scikit-learn-contrib/boruta_py. A sample of the dataset, which is 0.5% of the full dataset, which is about 11,656 lines of data are randomly selected to perform Boruta. This is because it is not possible for Boruta to run for the entire dataset, as it will take a very long time to perform 100 iterations. The Boruta will select important descriptors for model building. Finally, the remaining important features is split into train and test set, and undergo RF machine learning with the target, with 1100 number of trees and default hyperparameters. The code and hyperparameters used are attached in the appendix B7, C and D.

CHAPTER 4: RESULTS AND DISCUSSIONS

4.1 Introduction

This section includes the distribution of the target properties, training and testing result, error deviation and descriptor importance. Overall, the result is acceptable and has agreement with other researches. In the discussion section, we also discuss the possible explanation behind the machine learning prediction.

4.2 Results and Discussion

4.2.1 Distribution of the values of the target properties

Below shows the distribution of the target properties of HOMO, LUMO, first excited state and oscillator strength after processed that will be used to build the model of random forest. The range of HOMO are kept in the range of -4 to -8 eV, while the LUMO are kept in 0 to -4 eV. However, the values of first excited state remains intact. The values of oscillator strength are transformed to $\log_{10}$ scale. The values of 0 are removed from the dataset because it cannot be transformed to log scale. The original values of oscillator strength are also used in the calculation to compare to the log form. The total number of molecules in the dataset is 2331362.

	HOMO	LUMO	First excited state	Oscillator strength	Oscillator strength ($\log_{10}$)
Count	2331362	2331362	2331362	2331362	2331362
Mean	-6.394434	-1.056809	4.546506	0.03959515	-2.132754
Standard deviation	0.6242598	0.8096127	0.8074386	0.08234856	0.9730753
Min	-7.997426	-3.997352	0.06114843	1.0 e-6	-6.000000
25%	-6.824615	-1.578260	4.061629	0.001868	-2.728623
50%	-6.367464	-0.8789277	4.607788	0.008938	-2.048760
75%	-5.962014	-0.3864017	5.099629	0.040999	-1.387227
Max	-3.001416	-0.000000	7.311144	1.600913	0.2043677

Table 1: Statistics of the properties of the molecules in the dataset.

HOMO

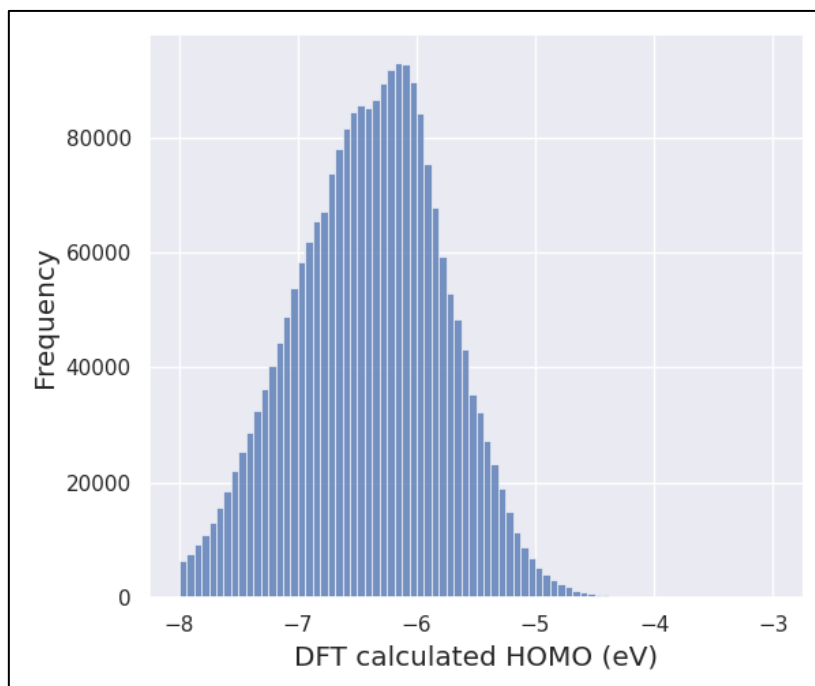


Figure 8: Distribution of the HOMO value in dataset.

LUMO

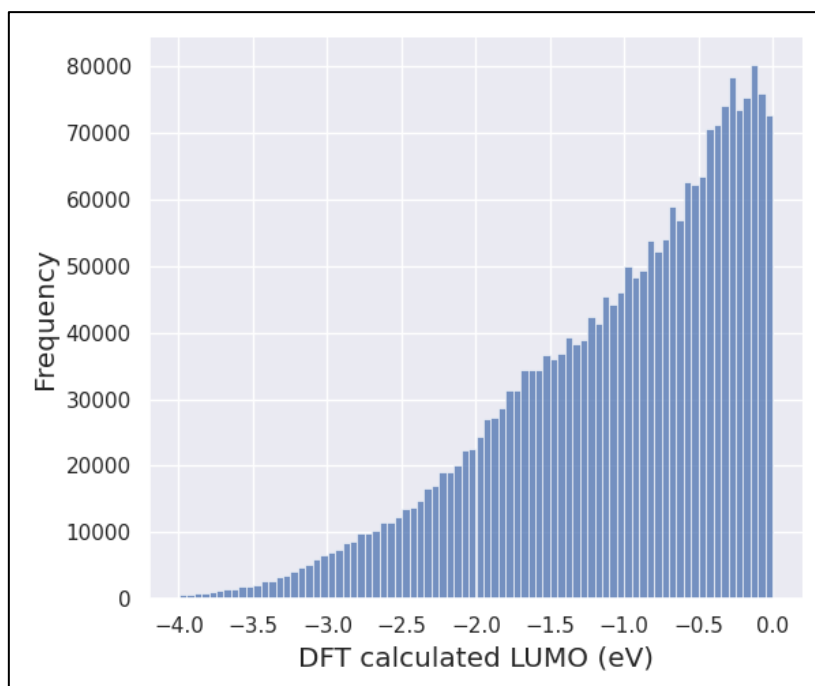


Figure 9: Distribution of the LUMO value in dataset.

First excited state

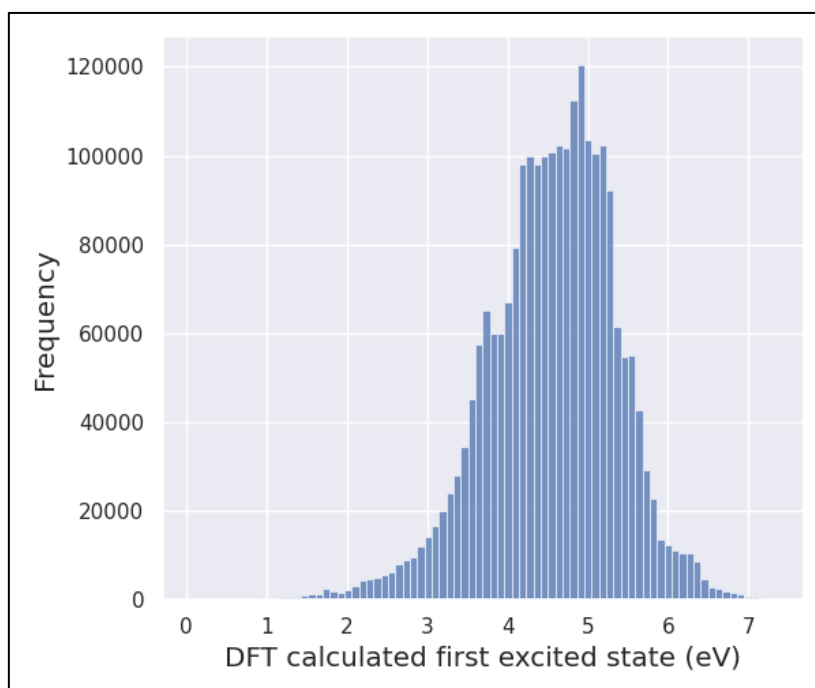


Figure 10: Distribution of the first excited state value in dataset.

Oscillator strength

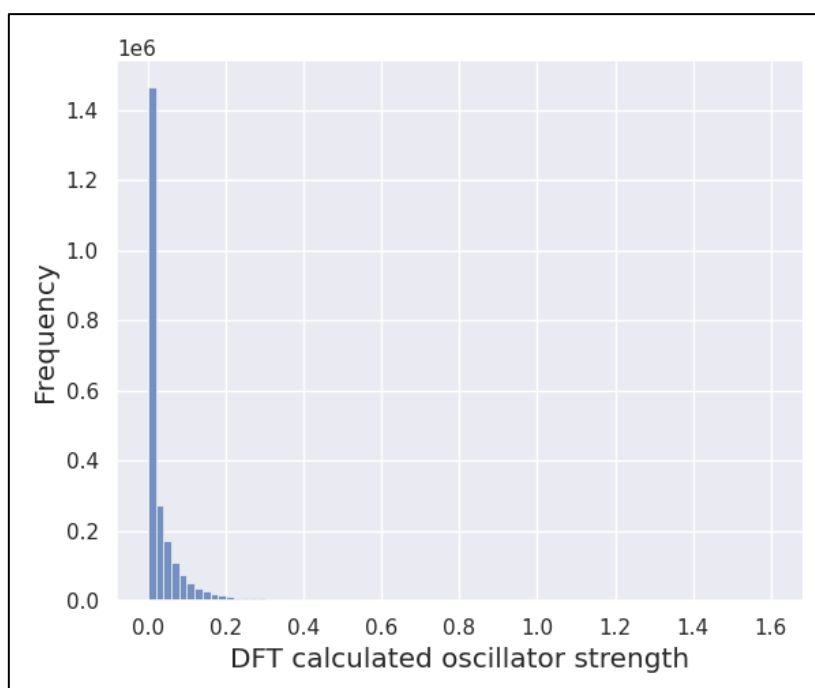


Figure 11: Distribution of oscillator strength in dataset

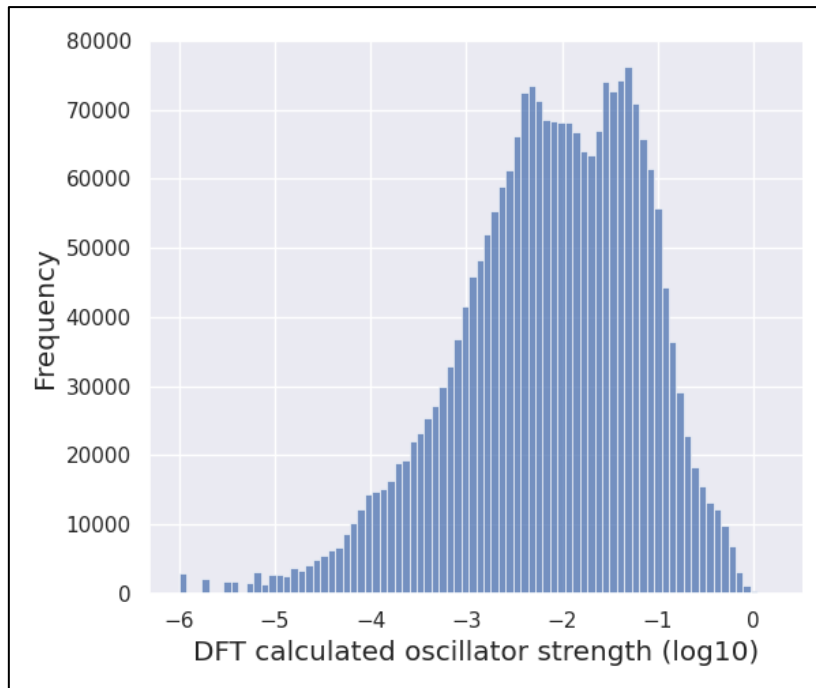


Figure 12: Distribution of the oscillator strength (log10) in dataset.

4.2.2 Training and testing result

The table 2 below shows the testing and training result using random forest regressor with the number of trees set to 1100. The number of trees is set to 1100 because when the number of trees increase, the more computational resources are needed. The maximum RAM can be used is 200GB, and the total memory usage during the calculation for each property is about 85% out of 200 GB. Higher number of trees will be resulting out of memory error.

	Training accuracy (R^2)	Testing accuracy (R^2)	Oob score	MAE (test model)	MSE (test model)	RMSE (test model)
HOMO	0.9845	0.8971	0.8968	0.1370 eV	0.0402 eV	0.2005 eV
LUMO	0.9913	0.9397	0.9394	0.1241 eV	0.0395 eV	0.1987 eV
First excited state	0.9891	0.9264	0.9264	0.1313 eV	0.0481 eV	0.2193 eV
Oscillator strength	0.9532	0.6606	0.6643	0.02199	0.0023	0.0480
Oscillator strength ($\log_{10}$)	0.9514	0.6737	0.6732	0.3802	0.3085	0.5554

Table 2: Result of the Random Forest Regression prediction.

The graphs with corresponding accuracies are plotted in scatter plots and heatmaps below. The orange lines show the best fit of the scatter plot. The heatmap distribution is plotted to show the distribution concentration of the values. From the heatmap we can see that although there are outliers in the scatter plot, we can see most of the predicted values concentrated along the diagonal, which indicates the predicted value match with the actual value.

1. HOMO

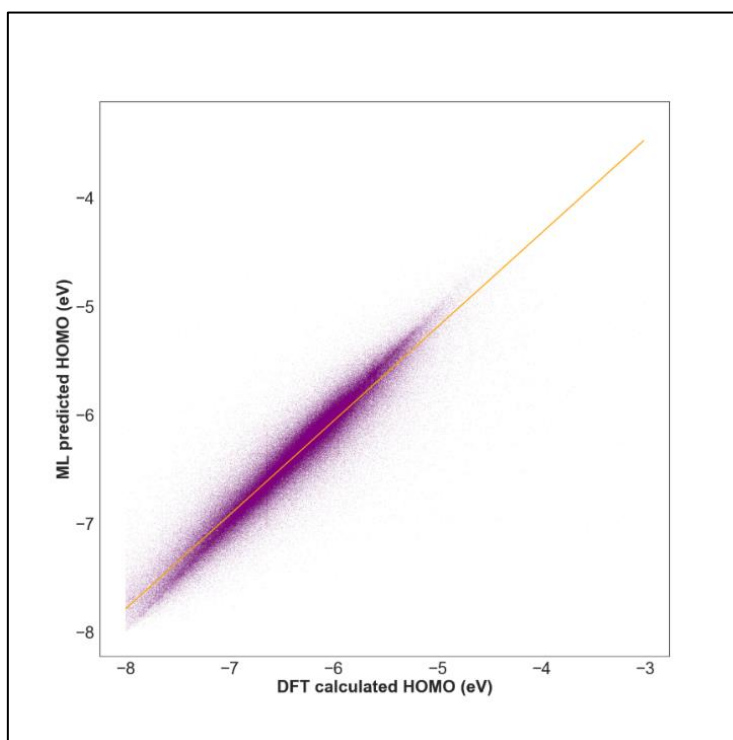


Figure 13: Testing result of HOMO with accuracy 0.8971

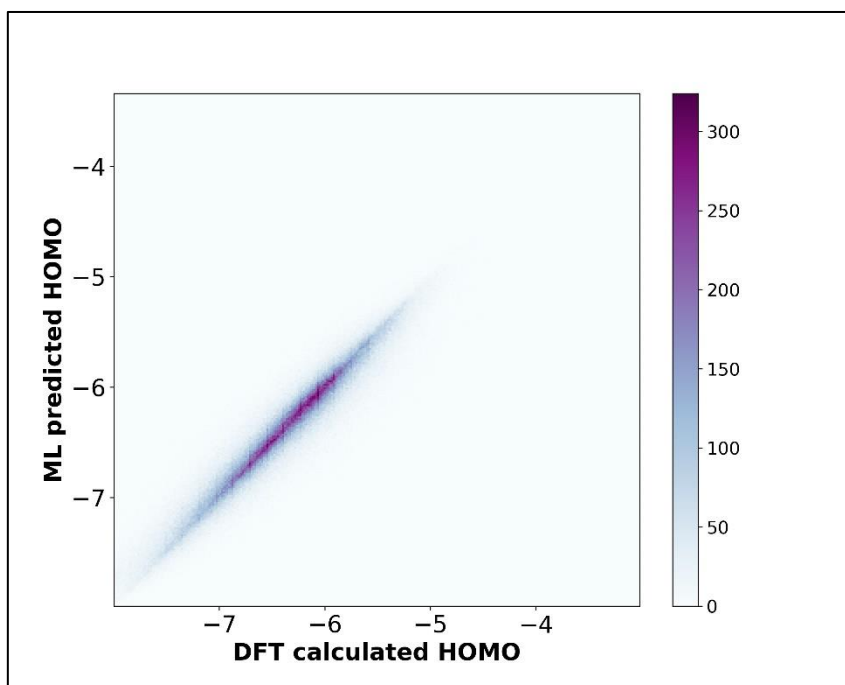


Figure 14: Heatmap distribution of the plot HOMO

2. LUMO

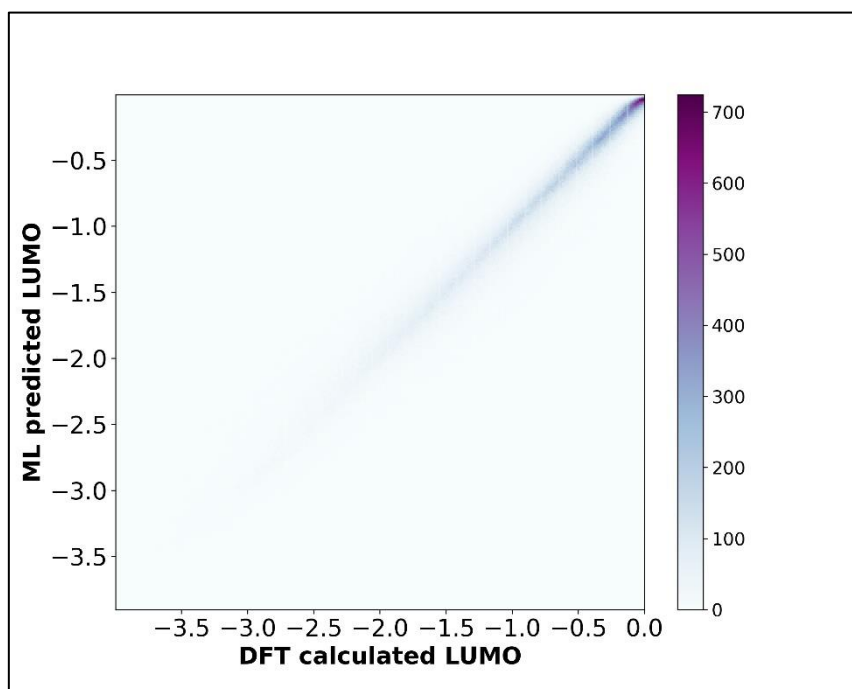


Figure 15: Testing result of LUMO with accuracy 0.9397

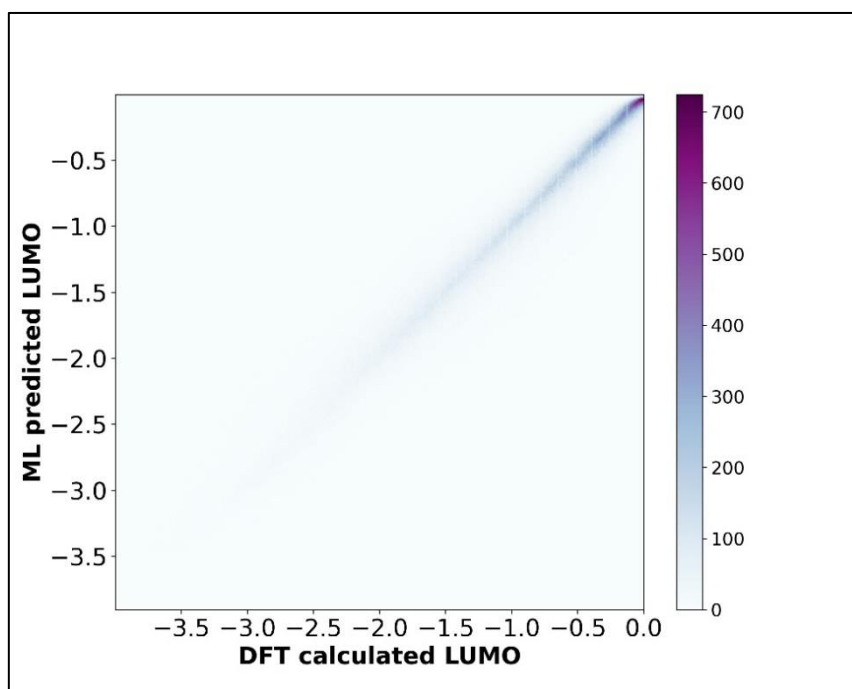


Figure 16: Heatmap distribution of the plot LUMO

3. First excited state

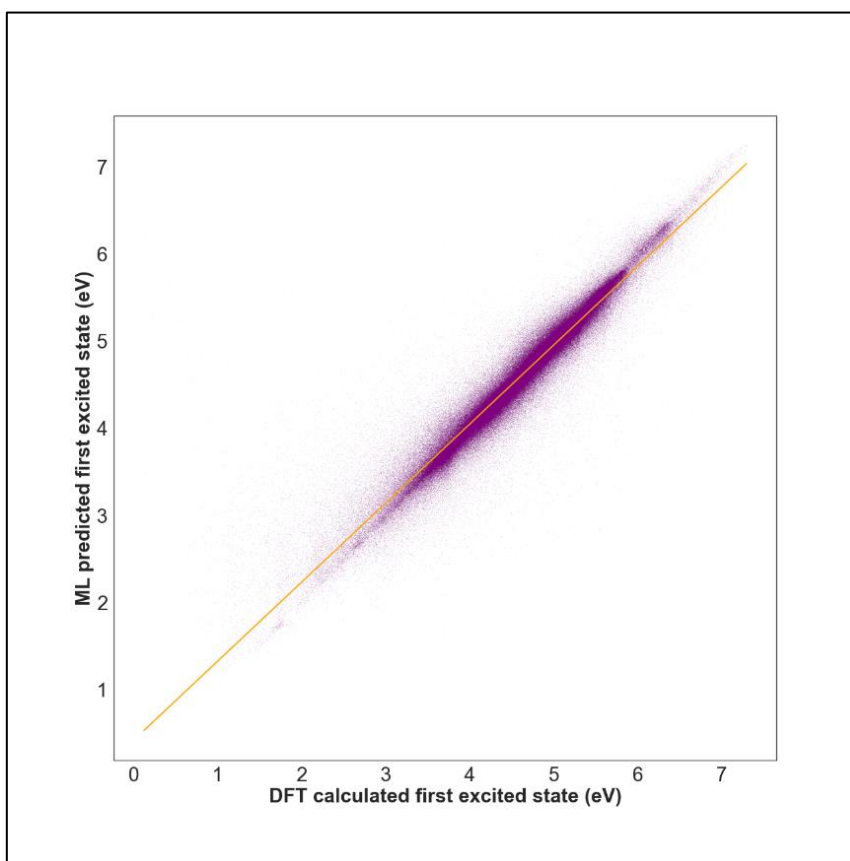


Figure 17: Testing result of first excited state with accuracy 0.9264

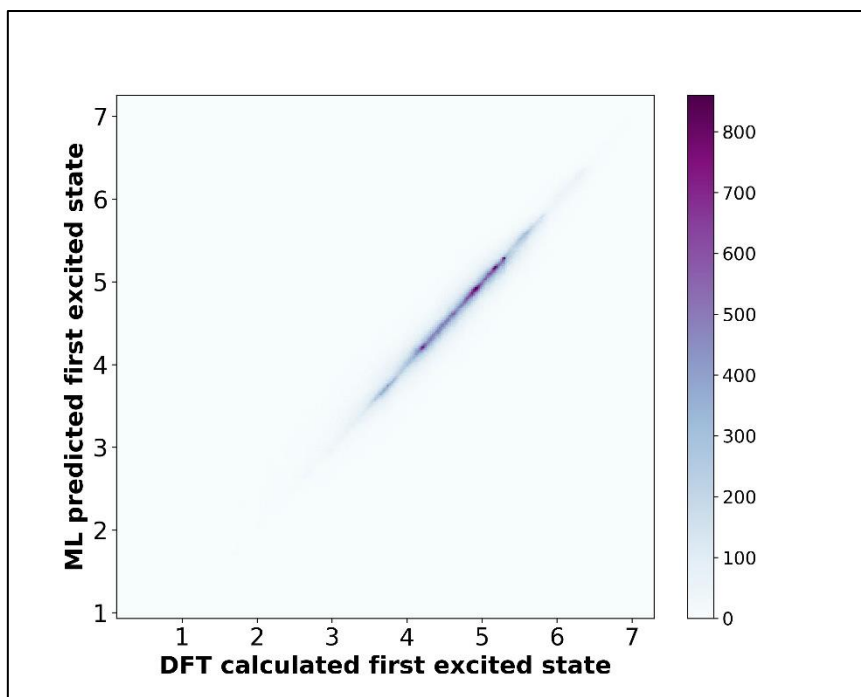


Figure 18: Heatmap distribution of the plot first excited state

4. Oscillator strength ($\log_{10}$)

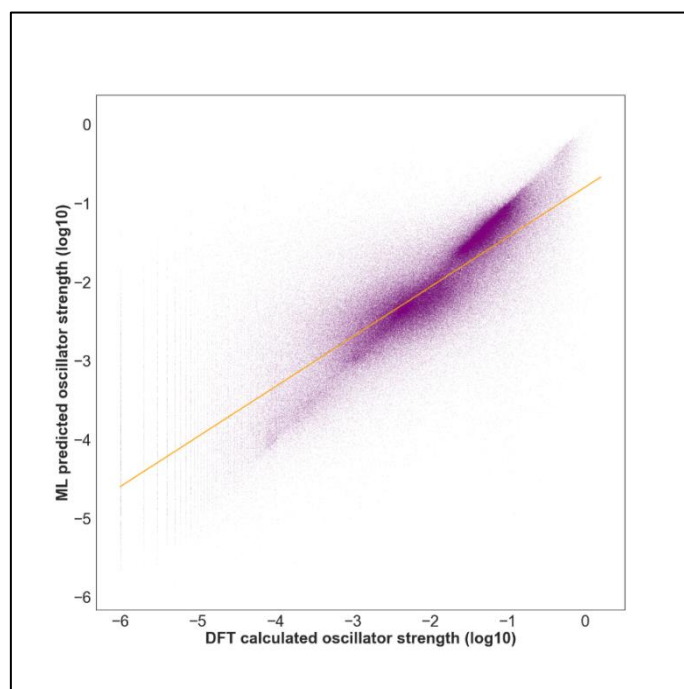


Figure 19: Testing result of oscillator strength with accuracy 0.6737

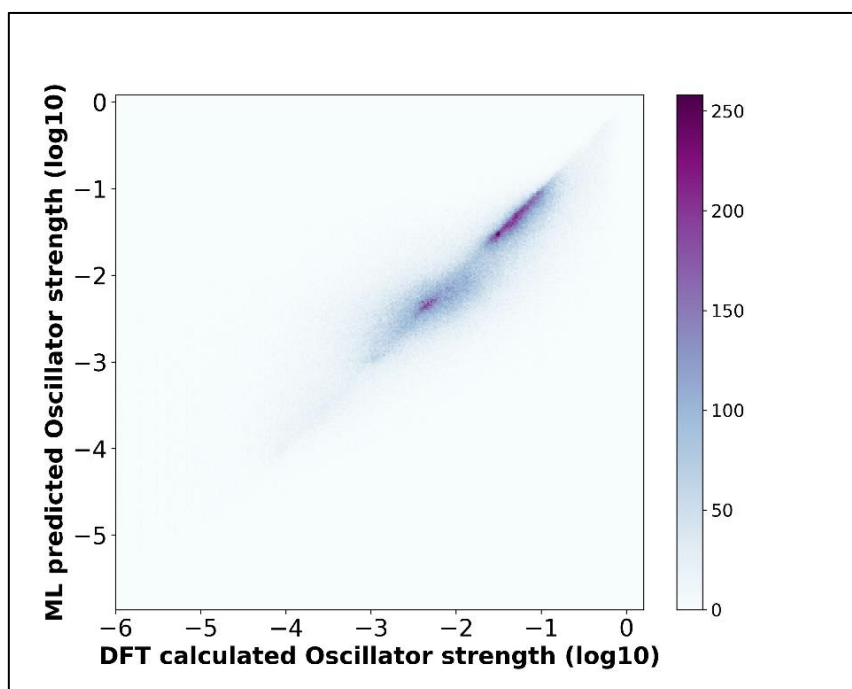


Figure 20: Heatmap distribution of the plot oscillator strength (log scale)

5. Oscillator strength

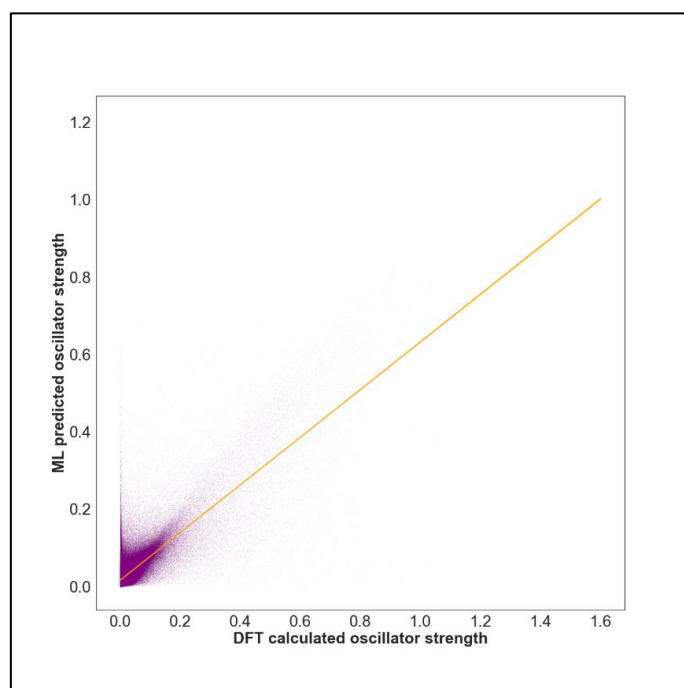


Figure 21: Testing result of oscillator strength with accuracy 0.6606

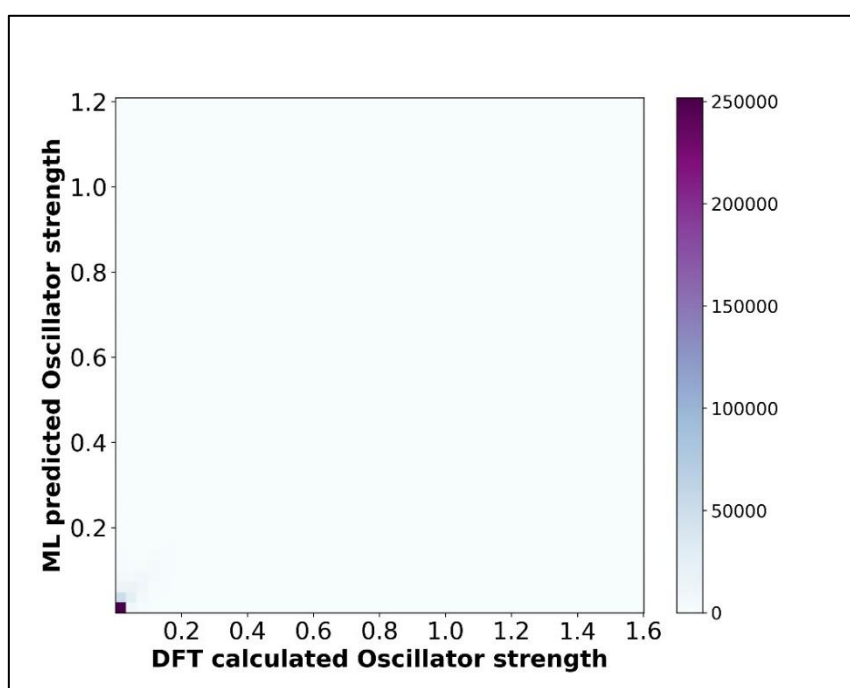


Figure 22: Heatmap distribution of the plot oscillator strength

4.2.3 Error deviation

We also measured the error deviation of our prediction data. The error deviation should be gaussian-like distribution. However, in this research, from the Table 3 we can see that the distribution exhibit skewness. The error deviation is the actual value minus the predicted value from the testing result. In this research, the error deviation graph of HOMO, LUMO first excited state, is limited to -1 to 1 eV. Oscillator strength in log form is limited to -2 to 2 in x-axis, while the original oscillator strength is limited to -0.1 to 0.1 in x-axis. There are very few outliers outside the range.

	Mean	Standard deviation
HOMO	-0.0004	0.2005
LUMO	-0.0005	0.1989
First excited state	0.0012	0.2191
Oscillator strength ($\log_{10}$)	0.0120	0.5553
Oscillator strength	-0.0014	0.0479

Table 3: Mean and standard deviation of the error deviation

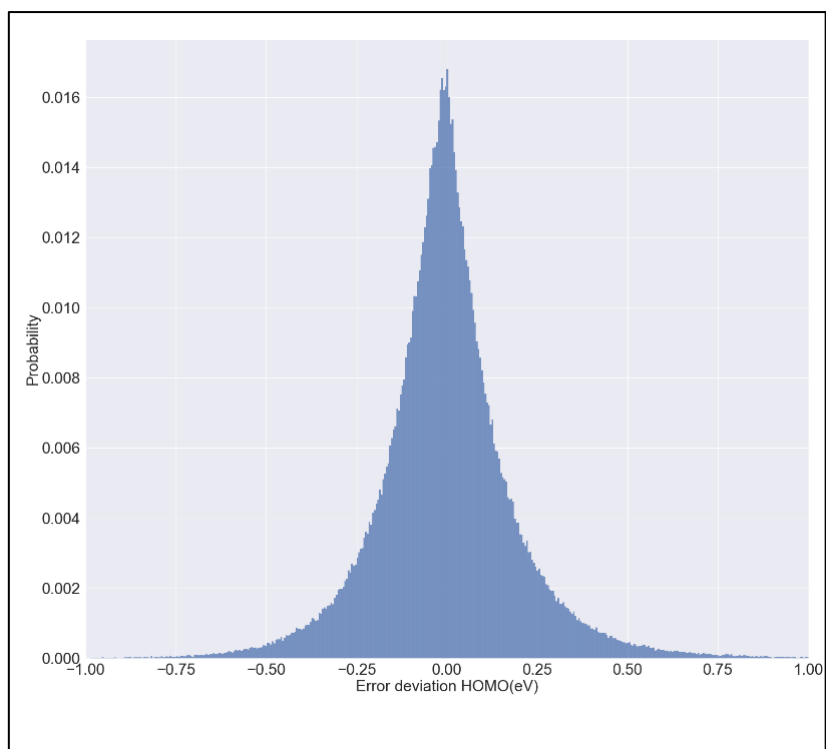


Figure 23: Error deviation of predicted HOMO value

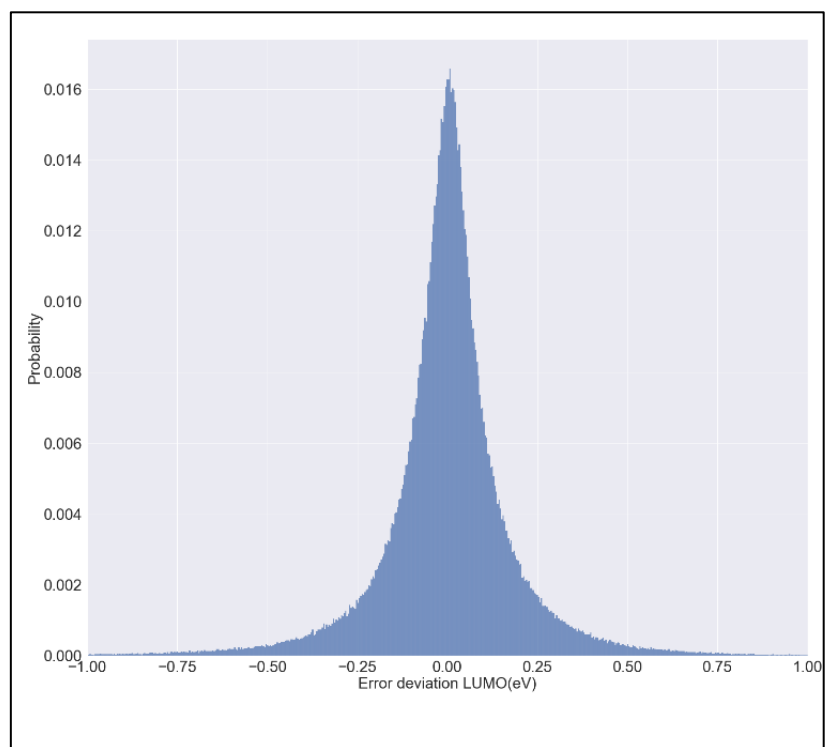


Figure 24: Error deviation of predicted LUMO value

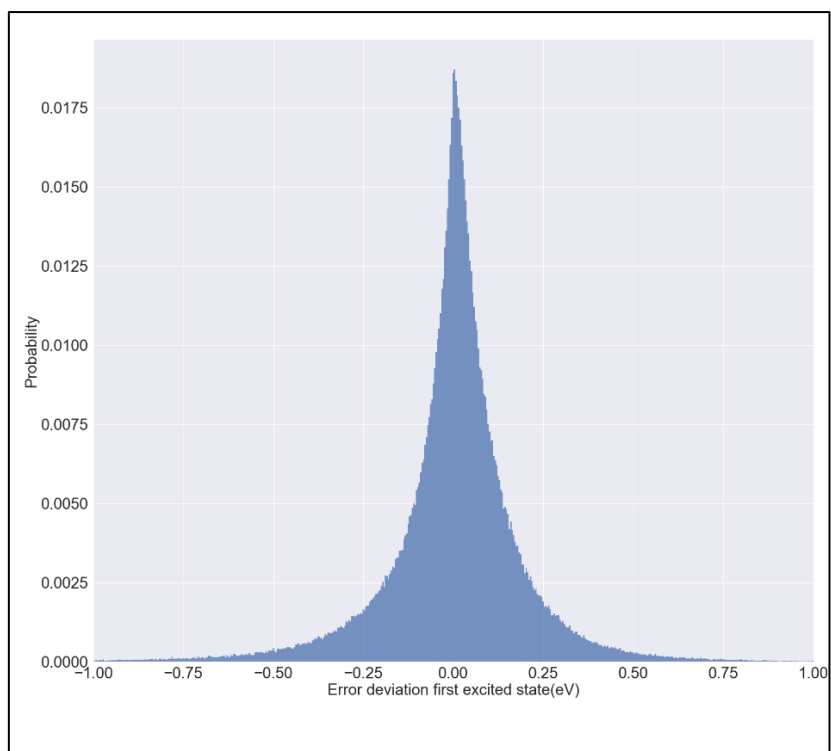


Figure 25: Error deviation of predicted first excited state value

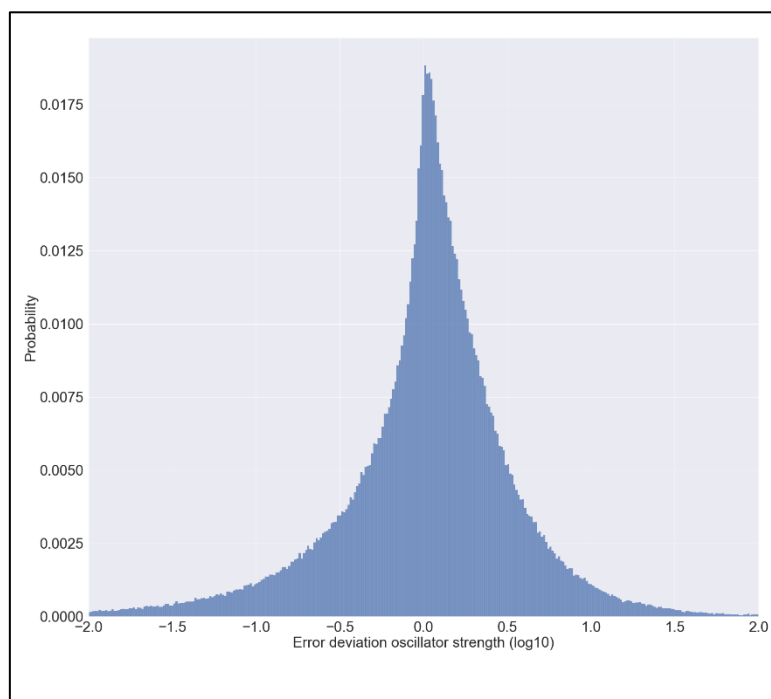


Figure 26: Error deviation of predicted oscillator strength (log scale) value

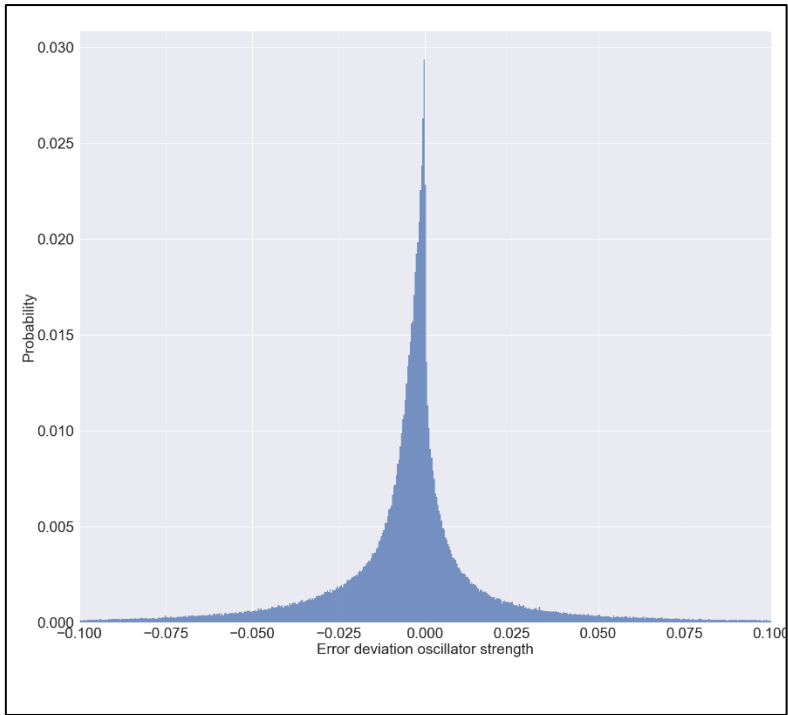


Figure 27: Error deviation of predicted oscillator strength value

4.2.4 Descriptors Importance

Tables in the appendix A shows the ranking of the descriptors importance along with their score. The top 5 descriptors for each property are listed below in the tables. The descriptors that are in bold are highly correlated with the target property. Thus, we can deduce that some of the highly correlated descriptors might contributed to the high ranking of the importance.

Those descriptors that are in bold are further investigated for their corresponding high correlated descriptors. This is because during the random forest fitting, we removed the some of the highly correlated descriptors first to save computation time. We want to deduce the descriptor can be substituted with their highly correlated counterparts. Those descriptors that are removed during the training to save computation are labeled as “dropped”. Some of the highly correlated descriptors with targets are also plotted in the graph below to show their relationship.

1. HOMO

Descriptor importance ranking	Descriptors	Score
HOMO	AATSC0s	0.180668
	fr_aniline	0.126874
	SsssN	0.050821
	SlogP_VSA1	0.037462
	RTpiPCTPC	0.024240

Table 4: Top 5 ranking descriptor importance of HOMO and their scores

Correlation of descriptors with target	Descriptors	Correlation
HOMO	FPsA1	0.439583
	AATS0s (dropped)	-0.468958
	AATSC0s	-0.468448
	ETAepsilon5	-0.456273

Table 5: Correlation scores of descriptors with target HOMO

Descriptor (HOMO)	Correlation between descriptors for selected descriptors	Correlation score
AATSC0s	AATS0s (dropped)	0.95606367
	ATSC0s (dropped)	0.916049437
	ETAepsilon4	0.843699278
	MAXDN2	0.823153012
	ATS0s (dropped)	0.820862256
	ETAepsilon2 (dropped)	0.818810319
	DELS (dropped)	0.814635569
	DELS2	0.80275951

Table 6: Descriptors correlation in HOMO

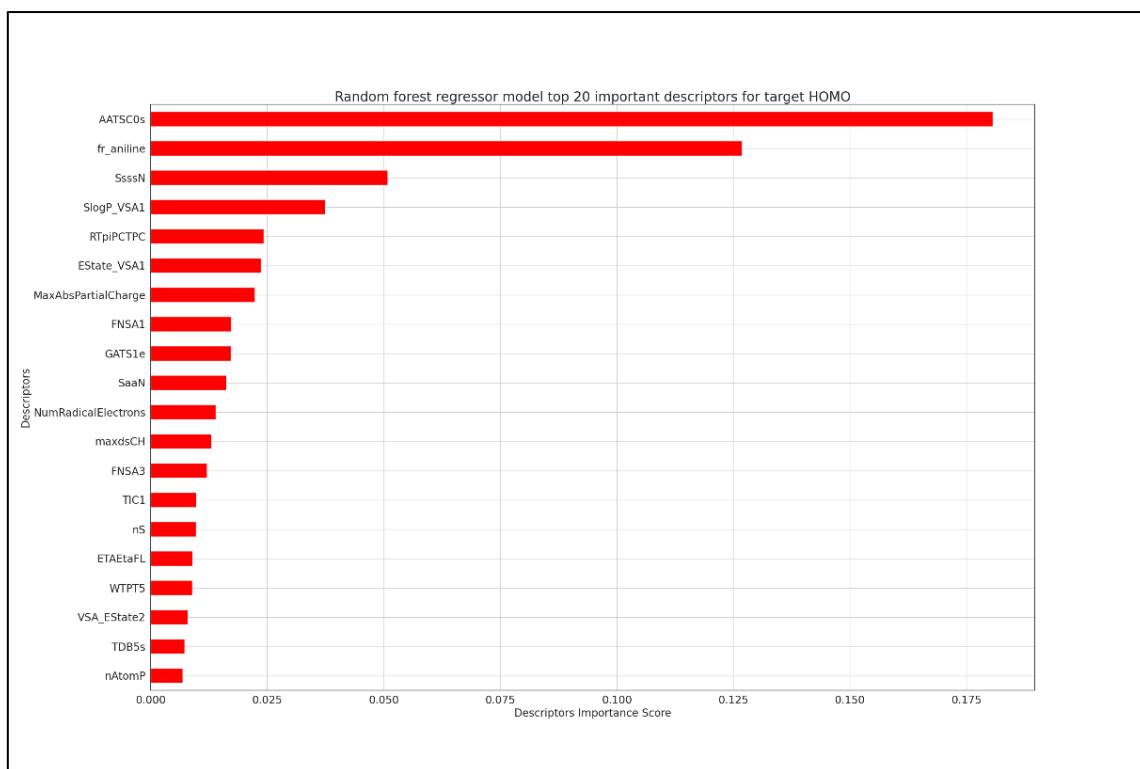


Figure 28: Top 20 descriptor ranking with target HOMO

2. LUMO

Descriptor importance ranking	Descriptors	Score
LUMO	GATS1m	0.320028
	NumRadicalElectrons	0.071456
	VSA_EState2	0.065339
	maxsOm	0.065152
	TDB2r	0.058543

Table 7: Top 5 ranking descriptor importance of LUMO and their scores

Correlation of descriptors with target	Descriptors	Correlation
LUMO	GATS1m	0.599412
	FractionCSP3(dropped)	0.598216
	GATS1v (dropped)	0.596402
	AATS0v (dropped)	-0.626125
	AATS2v (dropped)	-0.604069
	TDB2v (dropped)	-0.600846
	TDB2r	-0.608737

Table 8: Correlation scores of descriptors with target LUMO

Descriptor (LUMO)	Correlation between descriptors for selected descriptors	Correlation score
GATS1m	AATS1m (dropped)	0.85260665
	AMW (dropped)	0.852250937
	MIC2 (dropped)	0.839053796
	MIC3 (dropped)	0.836154273
TDB2r	AATS2v (dropped)	0.92953936

	AATS0v (dropped)	0.915785871
	GATS1v (dropped)	0.895691916
	AATS1v (dropped)	0.888126336
	TDB3r	0.88755875

Table 9: Descriptors correlation in LUMO

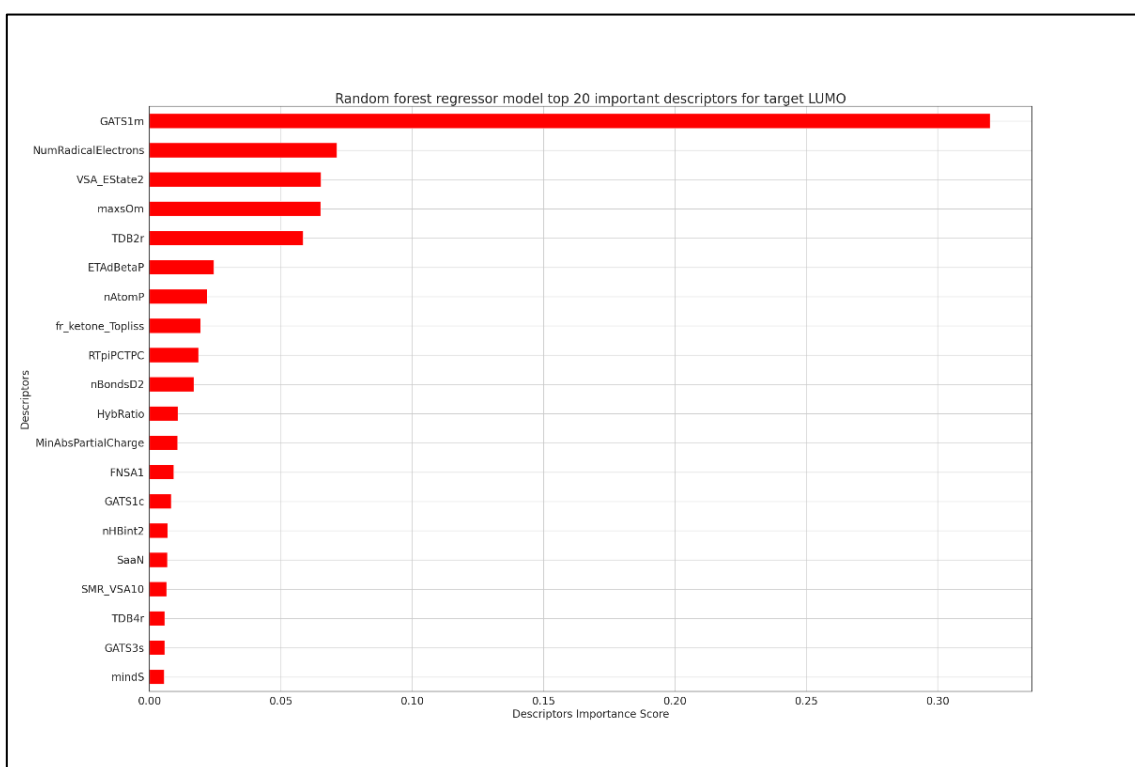


Figure 29: Top 20 descriptor ranking with target LUMO

3. First excited state

Descriptor importance ranking	Descriptors	Score
First excited state	nAtomP	0.252362
	NumRadicalElectron	0.072820
	fr_ketone_Topliss	0.070367
	nBondsD2	0.053637
	HybRatio	0.042032

Table 10: Top 5 ranking descriptor importance of first excited state and their scores

Correlation of descriptors with target	Descriptors	Correlation
First excited state	FractionCSP3(dropped)	0.535335
	HybRatio	0.515022
	nAtomP	-0.524224

Table 11: Correlation scores of descriptors with target first excited state

Descriptor (first excited state)	Correlation between descriptors for selected descriptors	Correlation score
HybRatio	FractionCSP3 (dropped)	0.982565676
	ETABetaPns (dropped)	0.937247554
	ETABetaP (dropped)	0.924200226
	ETABetans (dropped)	0.884371579
	ETAdEpsilonB (blue area)	0.878940247
	nwHBa (dropped)	0.873439851
	ETAEtaPL	0.870362071
nAtomP	ETABetans (dropped)	0.875672659
	nBondsM (dropped)	0.857840294

	nwHBa (dropped)	0.85527296
	ETABeta (dropped)	0.841955737
	HybRatio	0.840908291

Table 12: Descriptors correlation in first excited state

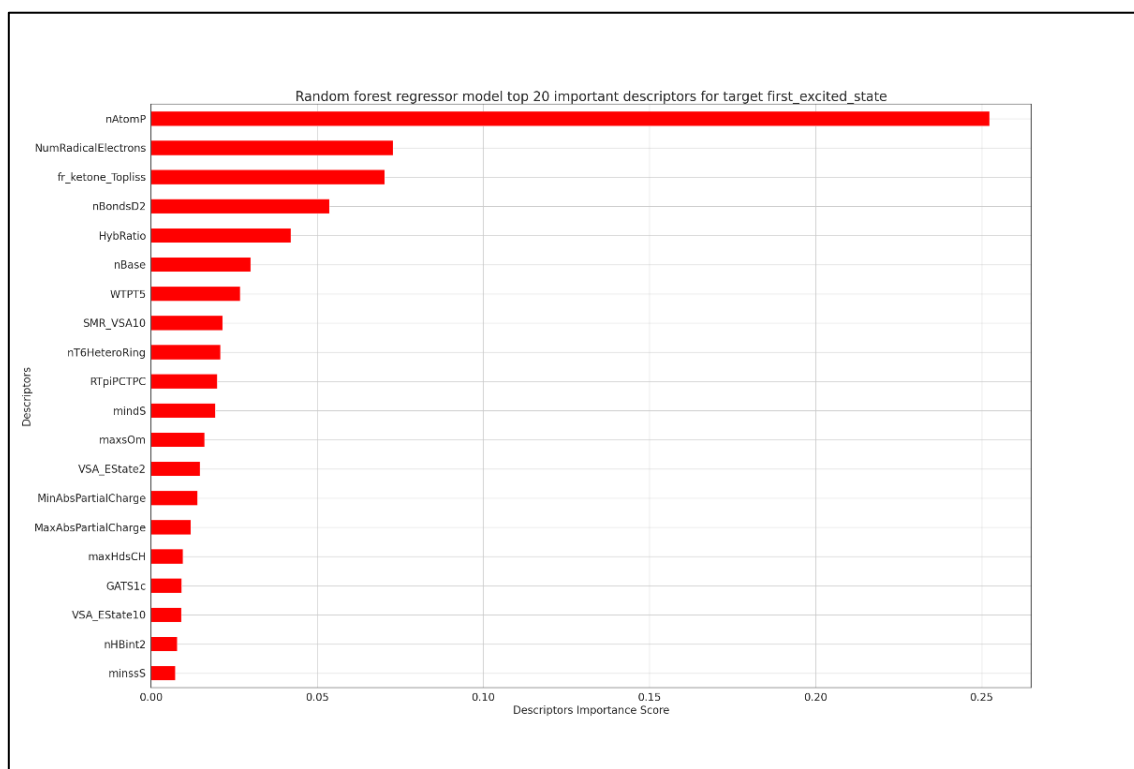


Figure 30: Top 20 descriptor ranking with target first excited state

4. Oscillator strength ($\log_{10}$)

Descriptor importance ranking	Descriptors	Score
Oscillator strength($\log_{10}$)	nAtomP	0.107404
	fr_ketone_Topliss	0.071655
	fr_C_O_noCOO	0.037435
	fr_aldehyde	0.032306
	MinAbsPartialCharge	0.028908

Table 13:Top 5 ranking descriptor importance of oscillator strength (log scale) and their scores

Correlation of descriptors with target	Descriptors	Correlation
Oscillator strength ($\log_{10}$)	TpiPC	0.307596
	nAromBond	0.29897038
	NumAromaticRings	0.294647487
	piPC6	0.294300407
	nAtomP	0.293782047
	naAromAtom	0.293577238
	fr_ketone_Topliss	-0.282783109
	fr_C_O	-0.288334448
	fr_ketone	-0.29717855
	fr_C_O_noCOO	0.293782047

Table 14: Correlation scores of descriptors with target oscillator strength in log scale

Descriptor (Oscillator strength $\log_{10}$)	Correlation between descriptors for selected descriptors	Correlation score
fr_C_O_noCOO	fr_C_O	0.871349434

nAtomP	nBondsM	0.857840294
	nwHBa	0.85527296
	ETABetans	0.875672659
	TpiPC	0.815949053
fr_ketone_Topliss	fr_ketone	0.937246105

Table 15: Descriptors correlation in oscillator strength of log scale

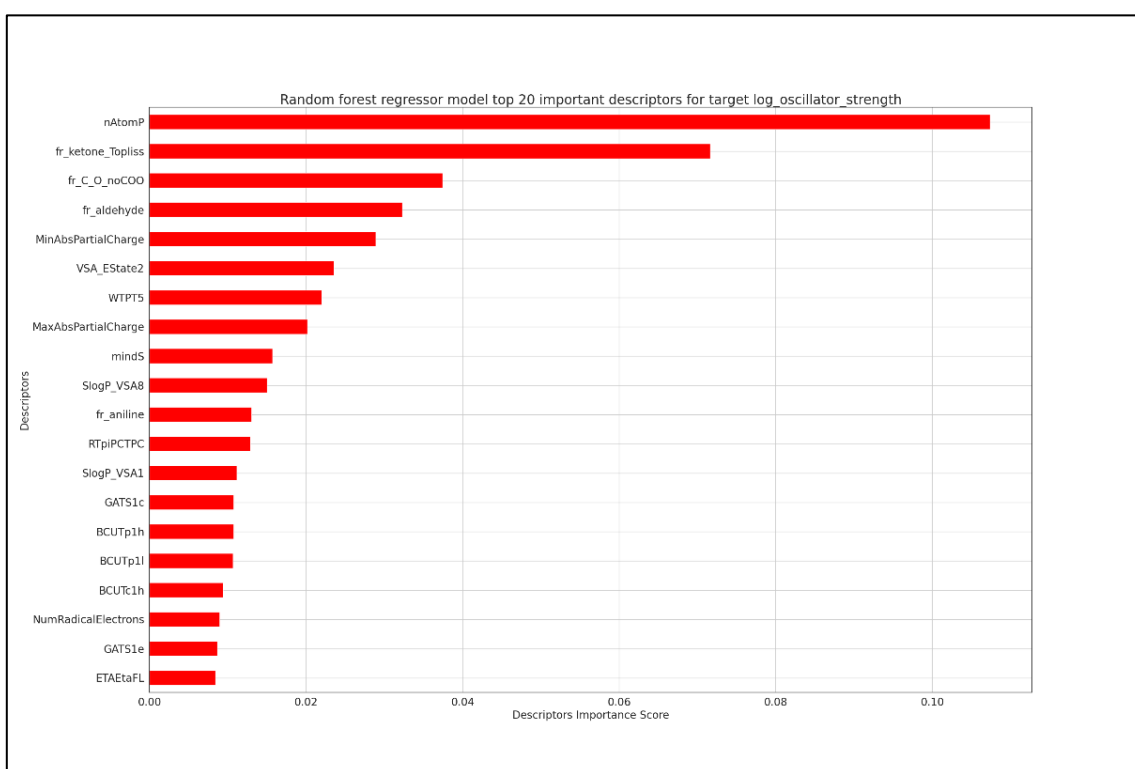


Figure 31: Top 20 descriptor ranking with target oscillator strength (log scale)

5. Oscillator strength

Descriptor importance ranking	Descriptors	Score
Oscillator strength	nAtomP	0.160646747
	SMR_VSA9	0.043282689
	MOMIYZ	0.04272173
	MinAbsPartialCharge	0.039037271
	RTpiPCTPC	0.037923811
	WTPT5	0.037762717
	SlogP_VSA8	0.030700999

Table 16: Top 5 ranking descriptor importance of oscillator strength and their scores

Correlation of descriptors with target	Descriptors	Correlation
Oscillator strength	RTpiPCTPC	0.385739619
	nAtomP	0.371932119
	SlogP_VSA8	0.349337999
	nBondsM	0.342723443
	HybRatio	-0.307030472
	FractionCSP3	-0.309506957

Table 17: Correlation scores of descriptors with target oscillator strength

Descriptor (Oscillator strength)	Correlation between descriptors for selected descriptors	Correlation score
nAtomP	nBondsM	0.857840294
	nwHBa	0.85527296
	ETABetans	0.875672659

	TpiPC	0.815949053
RTpiPCTPC	SlogP_VSA8	0.870962309
	NumAromaticRings	0.84640604
	nAromBond	0.861328302
	nBondsM	0.878256279
	ETABetans	0.816780141
	TpiPC	0.822387516
SlogP_VSA8	RTpiPCTPC	0.870962309

Table 18: Descriptors correlation in oscillator strength

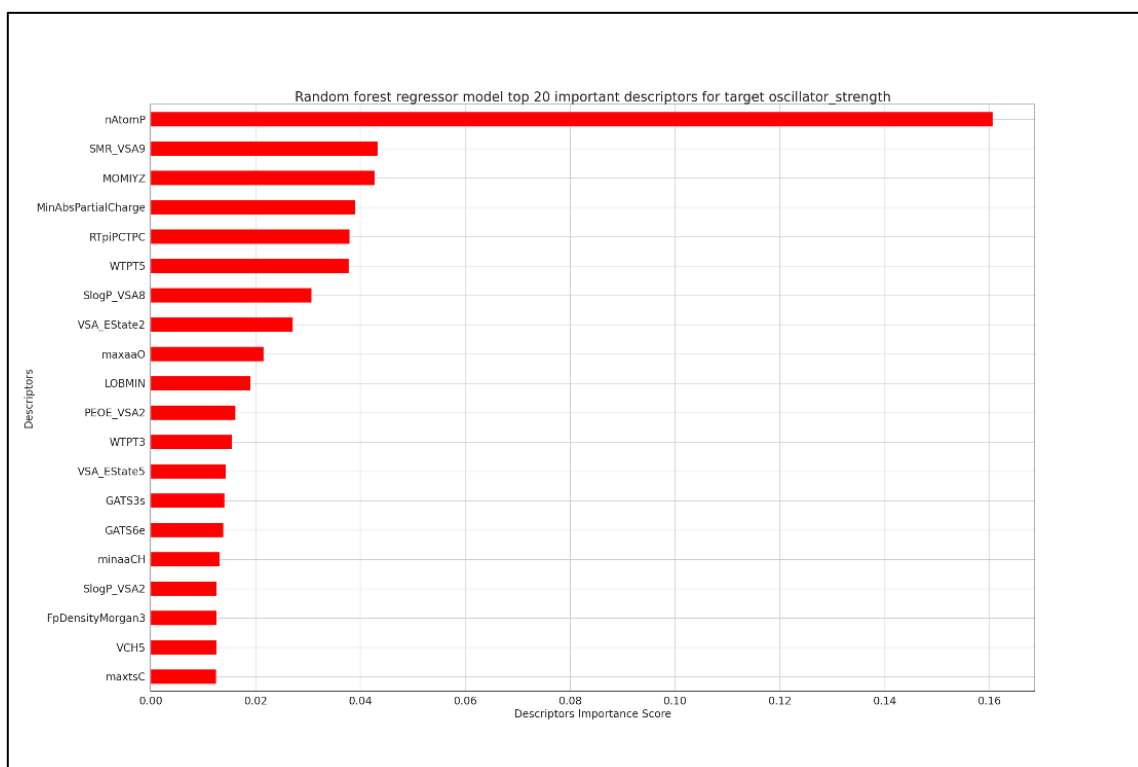


Figure 32: Top 20 descriptor ranking with target oscillator strength

4.2.5 Discussion

1. The application of Boruta python library

The Boruta runs random forests and generates random variables called 'shadow variables', which are based on existing variables, but the values are randomly shuffled. This will make sure that the 'shadow variables' do not have correlation with the target variable. In the variable importance run from the Boruta, some predictor variables might be better than the 'shadow variables'. Then, we want to pick the best 'shadow variable' to see if the predictor variables are better than the best 'shadow variable'. The predictors that score higher than the best 'shadow variables' are counted as hits that will be accepted to be the predictor. [26] In this research, the descriptors will be accepted if they are the hit predictor variables. The process is iterated 100 times and the results will go through a hypothesis test. Boruta will output the descriptors as confirmed, tentative, and rejected variables for every iteration. We can know that which descriptors are decided to keep and drop after performing Boruta.

2. Correlation threshold

In the research done by Lee (2020), the author used weak correlation and have low linear correlation between the descriptors. This indicates that they do not use the redundant and irrelevant features in their research. This can improve the performance of the machine learning model.[1] In our project, we put a threshold when the correlation greater than 0.90, the program will remove the descriptors that are correlated to each other. This can speed up the computation and save memory, since random forest model is very memory consuming. Therefore, after the calculation is done, we check the dropped highly correlated descriptors that might affect the result of our model.

3. RF regression determine feature importance method

We are using the default scikit-learn's feature importance ranking. The condition is based on how much each feature contributes to decreasing the weighed impurity. This is calculated as the mean and standard deviation of accumulation of the impurity decrease within each tree. For this research, the criterion used is the squared error. [27, 28]

4. Electronic properties of molecules

In the research done by Pereira, et al. (2017), they predicted a HOMO energy for the test set with MAE 0.15eV, and the RMSE of 0.21. For the LUMO energy level, they predicted a MAE of 0.16eV, and the RMSE of 0.23eV. Also, they have predicted the HOMO-LUMO gap with a MAE of 0.21eV and RMSE of 0.30eV. [2] In our research, we have predicted a HOMO energy level with about MAE 0.14 eV, RMSE of 0.2eV. While for the LUMO energy level, we have obtained MAE of 0.1241 eV and RMSE of about 0.2eV. We have also obtained a first excited state of MAE 0.1313eV and RMSE of 0.2193eV, although the first excited state does not necessarily equivalent to the HOMO-LUMO gap. The result shows that we have obtained the values of electronic properties similar to other research, and in the right direction. In the research done by Lee (2020), the author have also predicted a HOMO, LUMO and band gap with reaching coefficient of determination, R^2 of 0.85 and 0.80 in the train set and test set respectively using data of 135 non-fullerene acceptor/donor pairs based organic solar cells.[1]. In our research, we have achieved the R^2 of 0.89, 0.93 and 0.92 in predicting HOMO, LUMO, and first excited state respectively. This shows that by using a larger dataset, we have achieved a better result.

All the meaning of the descriptors can be found on the internet. AATSC0s is autocorrelation descriptor, which has the highest ranking in the HOMO prediction. In depth of studying the details of AATSC0s is not the focus in this study. We can assume that the autocorrelation descriptor as a dominant determinator of the HOMO level. From the research done by Woon, et al. (2021), they stated that the presence of nitrogen results in a deeper HOMO level. [29] In our research, we have found that the descriptors that contain nitrogen properties such as fr_aniline and SsssN contributed high importance to the model. The descriptor fr_aniline is the number of anilines, while the SsssN is the sum of atom type E-state of all tertiary nitrogens in molecules. The SlogP_VSA1 is the MOE logP VSA descriptor, while the RTpiPCTPC is the path count descriptor that calculates the ratio of total conventional bond order with total path count.

Similar as HOMO, while in LUMO energy levels the autocorrelation descriptors GATS1m and TDB2r has high ranking determining the LUMO energy level. We can see that the GATS1m and TDB2r descriptor also have the highest positive and negative correlation with the LUMO energy level.

GATS1m is the Geary autocorrelation of lag 1 weighted by mass, while TDB2r is the 3D topological distance-based autocorrelation - lag 2 / weighted by covalent radius. Electrotopological state descriptor also has high ranking in the LUMO energy level. VSA_EState2 is the hybrid EState-VSA descriptors and maxsOm is the maximum atom-type E-State of Oxygen with bonds. While for the NumRadicalElectrons, it describes the number of radical electrons the molecule has. This is match with the research done by Woon, et al.(2021), when they also state that the more electronegative the substituent atoms, the shallower the LUMO level. [29]

In the prediction of the value of first excited state, nAtomP has the highest importance ranking. The nAtomP shows the number of atoms in the largest pi system. It might because of the HOMO-LUMO gap, the magnitude of first excited state is inversely proportional to the extent of aromatic system. While the HybRatio is the fraction of sp³ carbons to sp² carbons. The fr_ketone_Topliiss is the number of ketones excluding diaryl. The nBondsD2 is the total number of double bonds, excluding the bonds to aromatic bonds.

5. Oscillator strength of molecules

For the research of predicting oscillator strength, our research has obtained a fitting result of 66%, and 67% in log scale. In the research done by Kang, et al. (2020), they have obtained a fitting result with R² of about 73% in oscillator strength test set. [30] Therefore, the result is compelling. The lower R² is also due to the fact that the data distribution of oscillator strength is not Gaussian-like or half-gaussian-like. The resulting plotting graph of prediction and actual value is also similar compared to their research. In their research, they stated that the nitrogen containing heterocycles play important roles in the inducing of high oscillator strength. It is also identified that the fragments that inducing high oscillator strength have many π conjugation bonds. In our research, we have found that the nAtomP contributes to high ranking both in oscillator strength value and the log scale in the random forest fitting. The descriptor refers to the number of atoms in the largest pi chain. According to quantum mechanics, the larger the pi chain, the easier the electron can be polarized by the oscillating electric field resulting in the excitation. The ease of polarizability is related to the oscillator strength.

Furthermore, Kang (2020) also stated that the most crucial molecular descriptor that determines the oscillator strength is the fraction of sp^3 carbon atoms.[30] However, in this research, we dropped the molecular descriptor FractionCSP3 before it runs random forest regression. They also stated the importance of κ -shape index, the molecular cyclicity and the existence of nitrogen connected aromatic rings to contribute to the high oscillator strength. They explained that this is because nitrogen atoms donate lone pair electrons, and they contribute to the oscillator strength by charge separation of molecule. In our research, we can see that the aromatic descriptor such as nAromBond (number of aromatic bonds), NumAromaticRings (number of aromatic rings for a molecule), naAromAtom (number of aromatic atoms) have high correlation with the oscillator strength in log form.

4.3 Summary

Random Forest along with molecular descriptors can be used to correlated the HOMO, LUMO, first excited state and oscillator strength of 2.3 million number of molecules tested with good accuracy. Overall, the result is compelling, and the values of accuracies are in common with the research papers. We can also see that how the correlation of the descriptors contributed to the ranking of importance of the prediction. Some can be seen as obvious that there is high correlation with the descriptors and the target properties. To achieve a better result, we need to learn a more robust technique to perform machine learning and interpolate skills to explain the result.

CHAPTER 5: CONCLUSION

5.1 Significant Findings

In this research, we have found that the use of machine learning with random forest is effective to predict the electronic properties and oscillator strength of the molecules. Random forest is an easy implementation of machine learning technique, and it provides us much information from the result. The output from the model can provide us the relationship between each descriptor and the target properties. We can further deduce the features that affect the result of prediction by analyzing the correlation between values.

We have also found that the high correlation between the descriptors and the target might also causing the high importance in the predictions. We can investigate that if a value of the target increases or decreases with the change of the value of the descriptor. Then using this technique, we can tune the chemical properties that we intended.

We have found that both the autocorrelation descriptors contributed high ranking to the prediction of HOMO and LUMO energy level. We have found the number of atoms in the largest pi system highly contributed to the prediction of oscillator strength. This might because of the presence of pi bonds lead to the polarizability and affecting the oscillator strength. A similar explanation can be applied to the contribution of number of largest pi systems determining the first excited state, where it contributed to the highest ranking of descriptor importance.

By using the bigger dataset, we might achieve a more accurate result than the other experiments. Although with larger dataset, there are more outliers and errors present, but the data that improve our prediction also increase.

5.2 Conclusion

In this research, we have demonstrated the big data prediction of the 2 million molecules using a machine learning model. We have successfully determined the performance of the machine learning to predict the HOMO, LUMO, first excited state and oscillator strength of the organic molecules. We have also found out the suitable descriptor to describe the molecules using the feature importance in machine learning. Then, we have also evaluated the correlation of the electronic structure of the organic molecules with the descriptors. As conclusion, random forest is one of the effective models to predict the value properties of the organic molecules. We can use this to tune and predict the molecules we want to investigate and apply in the research whether in the organic photovoltaic cells, or organic light emitting diode. This can save us time to search the suitable molecules in the vast chemical space. The correlation between the target properties and the descriptors is an important tool to investigate the relationship between them. The researcher can apply the correlation to tune the organic molecules to the properties desired. Further work should be done such as deploying the deep learning to predict the properties of the molecules at higher accuracy and machine learning model which is not limited by the requirement of Gaussian-like distribution.

5.3 Suggestion for Future Works

Although random forest is an easy and suitable algorithm to use, there are many machine learning method that we can try to apply to this research. However, deep learning methods are more computational intensive and need more skills to apply. There is not guarantee that the more intensive machine learning methods that can lead us to a more accurate result, but in this case random forest method is suitable because it is resistant to noise and deviations. The result shown in the research is good enough for a machine learning. The most important things to take care of is how to interpret the result, as machine learning do not provide us much information, which known as “black box”.

Furthermore, the solid knowledge about machine learning and molecular properties are also needed to perform this research correctly and identify mistakes. When interpreting machine learning model, the skills to interpret the results are essential. We have to know the concept of descriptors,

HOMO, LUMO, first excited state and oscillator strength before we start to perform the research. The skills of computer science to perform machine learning are also needed to build a good model of machine learning and performing data cleaning. We have to identify the correlation between the descriptors and target because correlation does not equal causation. What reason behind the machine learning random forest to fit perfectly is unknown if the correlation and causation of the data is not known.

To speed up the research, computer science knowledge is essential. If the computation can be done in parallel, the result will be faster. In this research, the hyperparameter number of jobs to run in parallel is set to -1 to make the random forest calculation done parallel using all 4 processors deployed. In the future attempt, we can even use more processors to increase efficiency. Furthermore, we can reduce the amount of data to run such as feature reduction if needed. However, running the tasks parallel does not indeed always leading us to better computation result.

REFERENCES

1. Lee, M.-H., *Robust random forest based non-fullerene organic solar cells efficiency prediction*. Organic Electronics, 2020. **76**: p. 105465.
2. Pereira, F., et al., *Machine Learning Methods to Predict Density Functional Theory B3LYP Energies of HOMO and LUMO Orbitals*. Journal of Chemical Information and Modeling, 2017. **57**(1): p. 11-21.
3. Kallmann, H. and M. Pope, *Positive Hole Injection into Organic Crystals*. The Journal of Chemical Physics, 1960. **32**(1): p. 300-301.
4. Kallmann, H. and M. Pope, *Bulk Conductivity in Organic Crystals*. Nature, 1960. **186**(4718): p. 31-33.
5. *The Electronic Structure of Organic Semiconductors*, in *Electronic Processes in Organic Semiconductors*. 2015. p. 1-86.
6. Shaw, J. and P. Seidler, *Organic electronics: Introduction*. IBM Journal of Research and Development, 2001. **45**: p. 3-9.
7. Sagiroglu, S. and D. Sinanc. *Big data: A review*. in *2013 International Conference on Collaboration Technologies and Systems (CTS)*. 2013.
8. Mjolsness, E. and D. Decoste, *Machine Learning for Science: State of the Art and Future Prospects*. Science (New York, N.Y.), 2001. **293**: p. 2051-5.
9. Himanen, L., et al., *Data-Driven Materials Science: Status, Challenges, and Perspectives*. Advanced Science, 2019. **6**(21): p. 1900808.
10. Agrawal, A. and A. Choudhary, *Perspective: Materials informatics and big data: Realization of the "fourth paradigm" of science in materials science*. APL Materials, 2016. **4**(5): p. 053208.
11. Hill, J., et al., *Materials science with large-scale data and informatics: Unlocking new opportunities*. MRS Bulletin, 2016. **41**(5): p. 399-409.
12. Wei, J., et al., *Machine learning in materials science*. InfoMat, 2019. **1**(3): p. 338-358.
13. Belgiu, M. and L. Drăguț, *Random forest in remote sensing: A review of applications and future directions*. ISPRS Journal of Photogrammetry and Remote Sensing, 2016. **114**: p. 24-31.

14. Cutler, A., D. Cutler, and J. Stevens, *Random Forests*. 2011. p. 157-176.
15. Consonni, V. and R. Todeschini, *Molecular Descriptors*, in *Recent Advances in QSAR Studies: Methods and Applications*, T. Puzyn, J. Leszczynski, and M.T. Cronin, Editors. 2010, Springer Netherlands: Dordrecht. p. 29-102.
16. Xue, L. and J. Bajorath, *Molecular descriptors in chemoinformatics, computational combinatorial chemistry, and virtual screening*. Comb Chem High Throughput Screen, 2000. **3**(5): p. 363-72.
17. Yang, Y., et al., *High-Performance Computing Based Big Data Analytics for Smart Manufacturing*. 2018. V003T02A013.
18. Fiore, S., M. Bakhouya, and W.W. Smari, *On the road to exascale: Advances in High Performance Computing and Simulations—An overview and editorial*. Future Generation Computer Systems, 2018. **82**: p. 450-458.
19. Puertas-Martín, S., et al., *Is high performance computing a requirement for novel drug discovery and how will this impact academic efforts?* Expert Opin Drug Discov, 2020. **15**(9): p. 981-986.
20. Huerta, E.A., et al., *Convergence of artificial intelligence and high performance computing on NSF-supported cyberinfrastructure*. Journal of Big Data, 2020. **7**(1): p. 88.
21. Goulding, A.N., J.J.F. Leung, and R.W. Neu. *Communicating Materials Systems Knowledge through Processing-Structure-Properties-Performance (PSPP) Maps*. 2018.
22. Hirata, S. and K. Shizu, *Organic light-emitting diodes: High-throughput virtual screening*. Nat Mater, 2016. **15**(10): p. 1056-7.
23. Cano, G., et al., *Automatic Selection of Molecular Descriptors using Random Forest: Application to Drug Discovery*. Expert Systems with Applications, 2016. **72**.
24. Rajan, K., *Materials Informatics: The Materials “Gene” and Big Data*. Annual Review of Materials Research, 2015. **45**(1): p. 153-169.
25. Schmidt, J., et al., *Recent advances and applications of machine learning in solid-state materials science*. npj Computational Materials, 2019. **5**(1): p. 83.
26. Nishida, K., *Finding Variable Importance with Random Forest & Boruta*. 2019.

27. Lewinson, E., *Explaining Feature Importance by example of a Random Forest*. 2019: towardsdatascience.com.
28. Pedregosa, *Feature importances with a forest of trees*. Journal of Machine Learning Research, 2011.
29. Woon, K.L., et al., *Relating molecular descriptors to frontier orbital energy levels, singlet and triplet excited states of fused tricyclics using machine learning*. Journal of Molecular Graphics and Modelling, 2021. **105**: p. 107891.
30. Kang, B., C. Seok, and J. Lee, *Prediction of Molecular Electronic Transitions Using Random Forests*. Journal of Chemical Information and Modeling, 2020. **60**(12): p. 5984-5994.

APPENDIX

Appendix A: Ranking of the descriptors importance

1. HOMO

1	AATSC0s	0.180668
2	fr_aniline	0.126874
3	SsssN	0.050821
4	SlogP_VSA1	0.037462
5	RTpiPCTPC	0.02424
6	EState_VSA1	0.023693
7	MaxAbsPartialCharge	0.02233
8	FNSA1	0.017314
9	GATS1e	0.017205
10	SaaN	0.016211
11	NumRadicalElectrons	0.013978
12	maxdsCH	0.013071
13	FNSA3	0.012078
14	TIC1	0.009785
15	nS	0.009687
16	ETAEtaFL	0.008982
17	WTPT5	0.008929
18	VSA_EState2	0.007994
19	TDB5s	0.007267
20	nAtomP	0.006822
21	PEOE_VSA1	0.0067
22	maxsssN	0.006633
23	SlogP_VSA2	0.006441
24	VCH7	0.006309
25	maxssNH	0.005924
26	SlogP_VSA8	0.005665
27	ndssC	0.005426
28	SMR_VSA3	0.005315
29	nBase	0.00502
30	VCH5	0.00499
31	nHBDonLipinski	0.004613
32	SHdsCH	0.004597
33	TDB1s	0.004523
34	VSA_EState4	0.004522
35	MinAbsPartialCharge	0.004483
36	BCUTc1h	0.004412
37	SCH7	0.00432
38	BIC0	0.003985
39	MIC0	0.003956
40	fr_allylic_oxid	0.003944
41	MAXDN2	0.003708

42	PEOE_VSA3	0.003482
43	SHCsatu	0.003236
44	TopoPSA	0.003202
45	nHBAcc3	0.003138
46	fr_NH1	0.003064
47	VP7	0.002998
48	GATS1s	0.002992
49	VSA_EState5	0.002842
50	GATS2e	0.002836
51	VSA_EState3	0.002808
52	TpiPC	0.0028
53	maxtN	0.002783
54	GATS2c	0.002735
55	maxaaN	0.002536
56	SHother	0.002527
57	SMR_VSA2	0.002472
58	GATS1c	0.002448
59	SlogP_VSA11	0.002397
60	fr_amide	0.002393
61	minaaCH	0.002393
62	maxaaNH	0.002392
63	SMR_VSA10	0.002363
64	TDB2s	0.002345
65	maxaaaC	0.002308
66	EState_VSA9	0.002302
67	AATS0m	0.002181
68	PEOE_VSA8	0.00216
69	nTRing	0.002131
70	VSA_EState9	0.002104
71	SdO	0.002082
72	minssS	0.002057
73	ETAdEpsilonC	0.002053
74	VSA_EState10	0.002027
75	maxaaCH	0.001998
76	hmax	0.00198
77	DPSA1	0.001978
78	ETAepsilon5	0.001889
79	GATS2s	0.001851
80	minHother	0.001826
81	ETAdEpsilonA	0.001818
82	BCUTp11	0.00179
83	MAXDP2	0.001717
84	nRing	0.001707
85	ETAdEpsilonB	0.001691
86	GATS3s	0.001662
87	ATSC0i	0.001635

88	PEOE_VSA2	0.001606
89	NumAliphaticHeterocycles	0.001592
90	TDB4s	0.001577
91	HallKierAlpha	0.001577
92	PEOE_VSA7	0.001551
93	MinAbsEStateIndex	0.001547
94	MAXDN	0.001537
95	AATSC0v	0.001523
96	WTPT3	0.001508
97	GATS1i	0.001497
98	SlogP_VSA4	0.001496
99	VSA_EState1	0.001484
100	SHBa	0.001482
101	RPCG	0.001478
102	maxHBint3	0.00146
103	MDEN22	0.001453
104	E1m	0.001416
105	RHSA	0.001411
106	EState_VSA7	0.00141
107	maxHdsCH	0.001344
108	WTPT4	0.001335
109	qed	0.001329
110	PEOE_VSA4	0.00131
111	EState_VSA6	0.001284
112	EEDzs	0.001267
113	ATSC0c	0.001242
114	BCUTp1h	0.001228
115	VSA_EState7	0.001194
116	HybRatio	0.001193
117	fr_Nhpyrrole	0.001177
118	maxssCH2	0.001171
119	maxdO	0.00117
120	ndsCH	0.001155
121	BalabanJ	0.001153
122	AMR	0.00115
123	RNCS	0.001144
124	AATSC0p	0.001134
125	SpMin2Bhi	0.001089
126	maxHssNH	0.001083
127	BIC1	0.001081
128	BCUTw1h	0.001078
129	maxsCH3	0.001065
130	SlogP_VSA5	0.00105
131	GATS2i	0.001044
132	GATS1m	0.001039
133	GATS4m	0.00103

134	ETAEtaF	0.001028
135	mindS	0.001014
136	VE3Dzp	0.001014
137	SlogP_VSA12	0.001009
138	E2m	0.001
139	FPSA2	0.000999
140	WNSA3	0.000999
141	RPCS	0.000998
142	NumAromaticHeterocycles	0.000998
143	SMR_VSA9	0.000983
144	DELS2	0.000983
145	PEOE_VSA6	0.000981
146	GATS3m	0.000968
147	EState_VSA8	0.000965
148	bpol	0.000961
149	Chi4n	0.00096
150	RNCG	0.00094
151	GATS4s	0.000938
152	fr_NH0	0.000932
153	VE3Dt	0.000932
154	E1v	0.000931
155	ETAShapeY	0.000926
156	GATS4v	0.000925
157	GATS5e	0.000924
158	ETAEtaPF	0.000924
159	TDB2r	0.000924
160	RDF30i	0.000923
161	FPSA3	0.000919
162	ETAdBetaP	0.000918
163	GATS4e	0.000917
164	LOBMIN	0.000913
165	ETAepsilon4	0.000908
166	ASP4	0.000907
167	RDF15m	0.000906
168	MolLogP	0.000896
169	TDB5e	0.000891
170	E2i	0.000887
171	SHBint4	0.000887
172	TDB4r	0.000883
173	FpDensityMorgan3	0.000877
174	L2i	0.000875
175	WNSA1	0.000874
176	GATS5s	0.000872
177	JGI4	0.000866
178	TDB6i	0.000862
179	GATS2p	0.000851

180	E2v	0.000847
181	TDB6s	0.000844
182	GATS6e	0.000838
183	mindsCH	0.000835
184	VE3D	0.000826
185	SHCsats	0.000821
186	EState_VSA2	0.000814
187	ETAEtaPFL	0.000814
188	GATS4p	0.000811
189	maxHBint5	0.00081
190	geomShape	0.000802
191	TDB3s	0.0008
192	GATS5m	0.000796
193	BIC5	0.000793
194	AVP4	0.000787
195	TDB7i	0.000787
196	TDB4m	0.000778
197	MDEN12	0.000777
198	IC2	0.000775
199	RDF15i	0.000768
200	TDB5m	0.000759
201	AATS6e	0.000756
202	VPC6	0.000755
203	TDB3m	0.000754
204	THSA	0.000748
205	SMR_VSA6	0.00074
206	ASP3	0.00074
207	TDB7s	0.000737
208	GATS3p	0.000734
209	Ti	0.000726
210	SpMax4Bhp	0.000725
211	ZMIC5	0.000724
212	SlogP_VSA3	0.000723
213	ETAEtaPL	0.000722
214	GATS1p	0.000719
215	maxtsC	0.000718
216	ASP5	0.000712
217	fr_ArN	0.000708
218	ASP6	0.000704
219	maxsNH2	0.000703
220	MDEC23	0.000699
221	IC5	0.000696
222	RotBFrac	0.000693
223	piPC10	0.000688
224	nBondsD2	0.000688
225	GATS3v	0.000688

226	JGT	0.000688
227	EState_VSA4	0.000687
228	AVP5	0.000677
229	MIC5	0.000672
230	ETABetaPnsd	0.000662
231	WPSA3	0.000661
232	MLogP	0.000661
233	ZMIC1	0.000657
234	JGI2	0.000656
235	SlogP_VSA10	0.000655
236	fr_Ar_OH	0.000644
237	minHssNH	0.000635
238	TDB6m	0.000634
239	SMR_VSA1	0.000634
240	P2i	0.000634
241	TDB6r	0.000631
242	WPSA1	0.000629
243	MOMIZ	0.000625
244	AVP6	0.000623
245	MDEO11	0.000622
246	SHBint5	0.000611
247	AVP7	0.000611
248	Ks	0.00061
249	MDEC22	0.000606
250	PEOE_VSA12	0.000605
251	MOMIYZ	0.000603
252	nHBAcc	0.000594
253	TPC	0.000577
254	Kier3	0.000577
255	TDB8s	0.00056
256	TDB7m	0.000558
257	WNSA2	0.000552
258	PEOE_VSA14	0.000548
259	ETAShapeP	0.000537
260	GATS8s	0.000537
261	WPSA2	0.000524
262	SsOH	0.000516
263	maxsOH	0.000506
264	minHsNH2	0.000497
265	nHBAccLipinski	0.000493
266	fr_C_O	0.000486
267	ATS0m	0.000478
268	nHeteroRing	0.000466
269	fr_benzene	0.000452
270	GATS8m	0.00045
271	TDB8m	0.00044

272	VCH4	0.000431
273	ATSC0p	0.000423
274	Ipc	0.000419
275	nHBa	0.000413
276	TDB9m	0.000404
277	ETAepsilon1	0.000388
278	maxHsNH2	0.000383
279	nAtomLC	0.000369
280	nT6Ring	0.000364
281	ETAPsi1	0.000329
282	ETAdPsiA	0.000325
283	Kier2	0.000321
284	MDEN33	0.000309
285	nsssN	0.0003
286	C1SP1	0.000262
287	fr_C_O_noCOO	0.000231
288	fr_bicyclic	0.00019
289	fr_phenol_noOrthoHbond	0.000117

2. LUMO

1	GATS1m	0.320028
2	NumRadicalElectrons	0.071456
3	VSA_EState2	0.065339
4	maxsOm	0.065152
5	TDB2r	0.058543
6	ETAdBetaP	0.024542
7	nAtomP	0.021941
8	fr_ketone_Topless	0.01947
9	RTpiPCTPC	0.018652
10	nBondsD2	0.016954
11	HybRatio	0.010783
12	MinAbsPartialCharge	0.010636
13	FNSA1	0.009183
14	GATS1c	0.008281
15	nHBint2	0.0069
16	SaaN	0.006748
17	SMR_VSA10	0.006441
18	TDB4r	0.005806
19	GATS3s	0.005724
20	mindS	0.005616
21	maxHdsCH	0.005563
22	maxtN	0.005029
23	VSA_EState1	0.004975
24	MaxAbsPartialCharge	0.004972
25	nT6HeteroRing	0.004957
26	GATS1i	0.004729
27	VCH5	0.00457
28	ETAdEpsilonB	0.004361
29	nBase	0.003973
30	GATS1s	0.003784
31	HallKierAlpha	0.00359
32	GATS1e	0.003583
33	fr_pyridine	0.003503
34	AATSC0s	0.003436
35	PEOE_VSA1	0.003363
36	MAXDN	0.003151
37	ETABetaPnsd	0.003098
38	PEOE_VSA10	0.003015
39	fr_aldehyde	0.002954
40	PEOE_VSA11	0.002919
41	AATSC0v	0.002735
42	SHCsatu	0.002696
43	GATS3m	0.002676
44	VSA_EState3	0.002577
45	minHCsat	0.002402

46	AATS0m	0.002398
47	bpol	0.002385
48	maxaasN	0.002335
49	SlogP_VSA3	0.00223
50	maxaaNH	0.002192
51	WTPT5	0.002186
52	TopoPSA	0.002166
53	ndssC	0.002082
54	SMR_VSA3	0.002076
55	PEOE_VSA3	0.001971
56	fr_NH0	0.001961
57	TDB3r	0.001915
58	VSA_EState5	0.001886
59	SlogP_VSA5	0.001885
60	SlogP_VSA4	0.001883
61	NumSaturatedHeterocycles	0.001759
62	BCUTw1h	0.001757
63	PEOE_VSA2	0.001707
64	SlogP_VSA1	0.001589
65	ETAEtaPF	0.001569
66	SlogP_VSA12	0.001566
67	VE3Dzs	0.001561
68	MDEO11	0.001543
69	BCUTp1l	0.001536
70	PEOE_VSA6	0.001535
71	GATS3e	0.001527
72	MDEN22	0.001489
73	GATS2e	0.001448
74	EState_VSA9	0.001371
75	VSA_EState4	0.001352
76	JGI3	0.001343
77	GATS1p	0.00133
78	maxHdCH2	0.001322
79	hmax	0.001299
80	SHdsCH	0.001283
81	nHBa	0.001252
82	FNSA3	0.00124
83	WTPT3	0.0012
84	RPCG	0.001168
85	TDB4m	0.001159
86	FPSA2	0.001156
87	TDB3s	0.001153
88	BalabanJ	0.001133
89	TDB2p	0.001118
90	maxssNH	0.001112
91	TDB5s	0.001106

92	GATS2m	0.001095
93	DPSA1	0.001089
94	VSA_EState10	0.001078
95	E1m	0.001071
96	TDB3m	0.001049
97	ETAepsilon5	0.001026
98	SMR_VSA6	0.001013
99	WNSA1	0.001008
100	fr_NH1	0.001004
101	SlogP_VSA2	0.001003
102	VSA_EState7	0.001003
103	nsssCH	0.000995
104	TDB1s	0.000966
105	PEOE_VSA8	0.000966
106	qed	0.000962
107	maxHssNH	0.000933
108	VSA_EState8	0.000928
109	RPCS	0.00092
110	MIC5	0.000918
111	ATSC0c	0.000911
112	RDF15p	0.000891
113	TDB5m	0.00089
114	RNCS	0.000868
115	SdO	0.000852
116	maxHCsats	0.000848
117	RNCG	0.000848
118	RDF30i	0.000844
119	GATS3i	0.000838
120	WNSA3	0.000826
121	ETAepsilon4	0.000821
122	E1v	0.000796
123	AATSC0p	0.000794
124	TDB5r	0.000778
125	TDB4s	0.00077
126	ETAdepsilonA	0.000767
127	Kier3	0.000764
128	ASP3	0.00076
129	BIC1	0.000759
130	ETAepsilon1	0.000752
131	TDB6r	0.000751
132	GATS2p	0.000741
133	VE3D	0.00074
134	maxHBint3	0.000737
135	fr_C_S	0.000733
136	TpiPC	0.000731
137	AVP5	0.000724

138	BCUTp1h	0.000721
139	TDB5e	0.000712
140	GATS6e	0.000691
141	LOBMIN	0.000689
142	TDB6s	0.000684
143	TDB7s	0.000684
144	L2i	0.000681
145	EState_VSA8	0.000681
146	ETAEtaPL	0.000679
147	TDB6e	0.000671
148	ETAShapeY	0.00067
149	BIC5	0.000664
150	ETAEtaPFL	0.00064
151	geomShape	0.000622
152	GATS6m	0.00062
153	EEDzs	0.000617
154	TDB2s	0.000611
155	BIC2	0.000608
156	TDB7i	0.000607
157	AVP0	0.000601
158	fr_C_O_noCOO	0.000593
159	ASP5	0.000589
160	JGI2	0.000588
161	TDB6m	0.000583
162	WPSA2	0.000577
163	IC2	0.000561
164	AVP7	0.000555
165	P2i	0.000552
166	AVP6	0.000549
167	JGT	0.000543
168	TDB7m	0.000526
169	MOMIYZ	0.000521
170	SHCsats	0.00052
171	nHBAcc	0.000514
172	MOMIZ	0.000502
173	RotBFrac	0.000492
174	RotBtFrac	0.000492
175	BIC0	0.000481
176	SPC6	0.000476
177	ASP7	0.000475
178	EState_VSA10	0.000474
179	ASP6	0.000473
180	VPC6	0.000465
181	NumSaturatedRings	0.000449
182	ATSC0p	0.000447
183	ATS5m	0.000444

184	SsOH	0.000441
185	TIC1	0.00044
186	VP2	0.00042
187	piPC10	0.000402
188	ETAEtaBRC	0.000398
189	maxdO	0.000396
190	naasN	0.000379
191	TDB9i	0.000377
192	TPC	0.000368
193	TIC0	0.000357
194	maxsNH2	0.00035
195	fr_C_O	0.000349
196	ATSC0i	0.000345
197	GRAV5	0.000342
198	GRAV4	0.000339
199	NumHAcceptors	0.000333
200	fr_imidazole	0.000318
201	nBondsS3	0.000293
202	nAtomLAC	0.000285
203	nHBDonLipinski	0.000248
204	maxdCH2	0.000236
205	C1SP3	0.000155
206	topoShape	0.000138
207	nHBAccLipinski	9.46E-05

3. First excited state

1	nAtomP	0.252362
2	NumRadicalElectrons	0.07282
3	fr_ketone_Topless	0.070367
4	nBondsD2	0.053637
5	HybRatio	0.042032
6	nBase	0.029976
7	WTPT5	0.026743
8	SMR_VSA10	0.021498
9	nT6HeteroRing	0.020841
10	RTpiPCTPC	0.019891
11	mindS	0.019377
12	maxsOm	0.016117
13	VSA_EState2	0.014721
14	MinAbsPartialCharge	0.014065
15	MaxAbsPartialCharge	0.011996
16	maxHdsCH	0.009531
17	GATS1c	0.009224
18	VSA_EState10	0.00914
19	nHBint2	0.007812
20	minssS	0.007313
21	maxsssN	0.006542
22	nT10Ring	0.006449
23	fr_aniline	0.006148
24	MDEN22	0.006061
25	SMR_VSA3	0.005951
26	VSA_EState4	0.005589
27	fr_Al_OH_noTert	0.005368
28	PEOE_VSA3	0.004857
29	PEOE_VSA2	0.004674
30	fr_nitroso	0.00457
31	SHdsCH	0.004288
32	maxHAvin	0.004219
33	PEOE_VSA10	0.003665
34	GATS1p	0.003605
35	fr_C_S	0.003505
36	PEOE_VSA11	0.003214
37	BCUTp1h	0.00318
38	maxssNH	0.003152
39	SsssN	0.003118
40	PEOE_VSA14	0.00305
41	GATS2c	0.003033
42	SlogP_VSA4	0.002991
43	GATS1e	0.002588
44	BCUTc1h	0.002584
45	VSA_EState3	0.002539

46	HallKierAlpha	0.002438
47	ETAdEpsilonB	0.002349
48	EState_VSA9	0.002309
49	SlogP_VSA2	0.002299
50	NumAliphaticRings	0.002223
51	VSA_EState1	0.002211
52	PEOE_VSA12	0.002186
53	GATS1s	0.002114
54	SdO	0.00209
55	SlogP_VSA12	0.002029
56	NumAliphaticHeterocycles	0.001945
57	GATS3s	0.001939
58	ETAEtaFL	0.001934
59	maxHCsats	0.00193
60	MAXDN2	0.001888
61	SlogP_VSA11	0.001859
62	ETAEtaPFL	0.001834
63	ETAdBetaP	0.001772
64	BCUTp1l	0.001761
65	maxHCsatu	0.001736
66	SlogP_VSA1	0.001677
67	WTPT4	0.001655
68	SlogP_VSA3	0.001653
69	GATS1i	0.001648
70	SHCsatu	0.001633
71	MAXDP2	0.001617
72	AATSC0p	0.001594
73	hmax	0.001577
74	fr_N_O	0.001577
75	VSA_EState5	0.001567
76	GATS3e	0.001553
77	fr_SH	0.001528
78	SHBa	0.001519
79	TpiPC	0.001476
80	PEOE_VSA8	0.001463
81	ndssC	0.001426
82	SpMin2Bhi	0.001411
83	RPCG	0.001404
84	WTPT3	0.001369
85	TopoPSA	0.001299
86	maxHdCH2	0.001279
87	qed	0.001276
88	ASP3	0.001272
89	RotBFrac	0.001262
90	VSA_EState8	0.001237
91	BalabanJ	0.00122

92	TDB2p	0.001194
93	VSA_EState7	0.001192
94	maxHssNH	0.001184
95	FPSA2	0.00117
96	GATS4s	0.001144
97	EEDzs	0.001131
98	EState_VSA8	0.001124
99	E1m	0.001116
100	AVP4	0.001106
101	ETAEtaPF	0.001106
102	VE3Dzs	0.001098
103	SMR_VSA9	0.001096
104	GATS3m	0.001095
105	maxHBint3	0.00109
106	RNCG	0.001085
107	MDEC33	0.001081
108	SlogP_VSA10	0.001075
109	VE3Dzp	0.001069
110	Kier3	0.001056
111	TDB1s	0.001053
112	VE3Dzi	0.001035
113	GATS2m	0.001022
114	GATS6s	0.001011
115	maxdsCH	0.001009
116	RotBtFrac	0.001009
117	P2i	0.001
118	GATS4e	0.000986
119	GATS4i	0.000962
120	GATS1m	0.000941
121	nHCsatu	0.000939
122	FPSA3	0.000925
123	BIC5	0.00092
124	JGI4	0.000913
125	ETAEtaPL	0.000913
126	RDF30i	0.000911
127	TDB3r	0.000904
128	WNSA1	0.000898
129	TDB2r	0.000891
130	TDB3m	0.000879
131	RDF25i	0.000869
132	TDB3s	0.000864
133	nT6Ring	0.000862
134	TDB5e	0.000851
135	TDB4s	0.000851
136	ETAShapeY	0.000844
137	FNSA1	0.000836

138	minHssNH	0.000834
139	TDB4r	0.00082
140	maxdsN	0.000806
141	ASP5	0.000803
142	EState_VSA10	0.000795
143	TDB5s	0.000791
144	AATS6i	0.000779
145	WNSA3	0.000774
146	AMR	0.000772
147	WPSA1	0.000767
148	SC5	0.000765
149	mindsCH	0.000764
150	SHBint5	0.000759
151	FNSA3	0.000759
152	fr_allylic_oxid	0.000758
153	ETAdPsiA	0.000746
154	piPC10	0.00074
155	AATSC0s	0.000739
156	SPC6	0.000731
157	TDB5r	0.000731
158	TDB7s	0.000726
159	ndsCH	0.000724
160	JGT	0.000722
161	TDB6r	0.000721
162	maxsOH	0.000719
163	TDB6s	0.000714
164	BIC1	0.000708
165	MIC5	0.000689
166	ASP6	0.000684
167	Chi4n	0.000683
168	AATS0m	0.000683
169	AATSC0v	0.000683
170	maxHBint5	0.000651
171	DPSA1	0.000647
172	TDB5m	0.000642
173	maxsNH2	0.000636
174	nFRing	0.000636
175	TDB7m	0.000634
176	TDB6m	0.00063
177	SHCsats	0.000626
178	SlogP_VSA5	0.000622
179	TDB8m	0.000621
180	nHBDonLipinski	0.000613
181	GGI4	0.000607
182	ASP7	0.000604
183	maxHsNH2	0.000597

184	EState_VSA1	0.000568
185	ETAShapeP	0.000565
186	AVP0	0.000556
187	TIC1	0.000554
188	MDEO12	0.000551
189	SsOH	0.000547
190	VP2	0.00054
191	maxdCH2	0.000539
192	SMR_VSA6	0.000521
193	nTRing	0.000509
194	bpol	0.000498
195	TPC	0.000482
196	ETAEtaBRC	0.000435
197	MDEO11	0.00041
198	nRotBt	0.00018
199	JGI9	0.000136

4. Oscillator strength ($\log_{10}$)

1	nAtomP	0.107404
2	fr_ketone_Topless	0.071655
3	fr_C_O_noCOO	0.037435
4	fr_aldehyde	0.032306
5	MinAbsPartialCharge	0.028908
6	VSA_EState2	0.023557
7	WTPT5	0.022002
8	MaxAbsPartialCharge	0.020199
9	mindS	0.015713
10	SlogP_VSA8	0.015025
11	fr_aniline	0.013065
12	RTpiPCTPC	0.012893
13	SlogP_VSA1	0.011147
14	GATS1c	0.010777
15	BCUTp1h	0.010757
16	BCUTp1l	0.010683
17	BCUTc1h	0.0094
18	NumRadicalElectrons	0.008971
19	GATS1e	0.008702
20	ETAetaFL	0.008427
21	MAXDN	0.008382
22	SHother	0.008042
23	maxaaCH	0.007923
24	GATS4s	0.007912
25	VSA_EState4	0.007493
26	P2i	0.007409
27	VSA_EState3	0.007404
28	minHother	0.007302
29	SlogP_VSA2	0.007191
30	VCH7	0.006979
31	PEOE_VSA8	0.006729
32	PEOE_VSA2	0.006689
33	RNCS	0.006305
34	MDEC23	0.006299
35	PEOE_VSA3	0.006165
36	VE3Dzs	0.006162
37	SHdsCH	0.006137
38	E1i	0.006089
39	SCH7	0.00607
40	GATS2s	0.00595
41	E2p	0.005936
42	SpMin2Bhi	0.005887
43	VE3Dzp	0.005814
44	hmax	0.005643
45	TDB6i	0.00561

46	qed	0.005591
47	ASP3	0.005589
48	RPCG	0.005426
49	WTPT4	0.005419
50	SMR_VSA10	0.005356
51	maxdO	0.005329
52	PEOE_VSA1	0.005302
53	GATS1i	0.005206
54	VE3D	0.005175
55	BIC5	0.005119
56	AVP5	0.005093
57	GATS1s	0.005086
58	RDF30i	0.005081
59	JGI5	0.005069
60	maxssNH	0.005051
61	geomShape	0.005046
62	TDB5s	0.005014
63	FPSA3	0.004946
64	TDB7i	0.004897
65	SdO	0.00488
66	IC5	0.004838
67	ETAEtaPF	0.004768
68	RNCG	0.004691
69	TDB4r	0.004618
70	ASP5	0.004561
71	PEOE_VSA6	0.004547
72	TDB4s	0.00454
73	HallKierAlpha	0.004478
74	VPC6	0.004395
75	minaaCH	0.004376
76	WPSA1	0.004375
77	TDB4m	0.004375
78	RDF25p	0.004374
79	TpiPC	0.004294
80	RDF15i	0.004278
81	HybRatio	0.00423
82	TDB2p	0.004176
83	SHCsatu	0.004175
84	SpMax4Bhp	0.004072
85	FPSA2	0.004069
86	TDB1s	0.00405
87	minHBint4	0.003958
88	maxHCsats	0.003918
89	IC2	0.00387
90	RDF15p	0.003817
91	BIC2	0.0038

92	maxHCsatu	0.003797
93	SlogP_VSA12	0.003796
94	SlogP_VSA5	0.003771
95	TDB2r	0.003713
96	ETAEtaF	0.003707
97	ETAdBetaP	0.003692
98	SHBint4	0.003689
99	ndssC	0.003573
100	AMR	0.00353
101	EEDzs	0.003507
102	naasC	0.003363
103	ATS5m	0.003361
104	ETAepsilon5	0.003313
105	SpMax5Bhi	0.003303
106	ATS4m	0.003254
107	SHCsats	0.003221
108	GATS1p	0.003169
109	minHCsatu	0.003162
110	VR3Dzp	0.003136
111	BIC1	0.003132
112	AVP0	0.003078
113	AATSC0v	0.003075
114	VR3Dzs	0.003068
115	maxdsCH	0.002944
116	ETABetaPnsd	0.002939
117	ETAdEpsilonB	0.002879
118	ETAEtaPFL	0.002842
119	ETAEtaPL	0.002842
120	fr_allylic_oxid	0.002744
121	SMR_VSA6	0.002698
122	piPC10	0.002607
123	EState_VSA6	0.00247
124	TIC1	0.002411
125	fr_C_O	0.002365
126	Ipc	0.002341
127	EState_VSA7	0.002329
128	SssO	0.002244
129	GRAV4	0.002188
130	GRAV5	0.002158
131	bpol	0.0021
132	maxsOm	0.002094
133	TPC	0.001943
134	NumAliphaticHeterocycles	0.001824
135	maxHAvin	0.001783
136	maxHdsCH	0.001743
137	TIC0	0.001596

138	SMR_VSA1	0.00142
139	nHeteroRing	0.001267
140	nBondsD2	0.001257
141	fr_nitro_arom_nonortho	0.00092
142	fr_C_S	0.000537
143	fr_thiophene	0.000243

5. Oscillator Strength

1	nAtomP	0.160647
2	SMR_VSA9	0.043283
3	MOMIYZ	0.042722
4	MinAbsPartialCharge	0.039037
5	RTpiPCTPC	0.037924
6	WTPT5	0.037763
7	SlogP_VSA8	0.030701
8	VSA_EState2	0.027155
9	maxaaO	0.021522
10	LOBMIN	0.019105
11	PEOE_VSA2	0.016153
12	WTPT3	0.015517
13	VSA_EState5	0.014363
14	GATS3s	0.014091
15	GATS6e	0.01389
16	minaaCH	0.013169
17	SlogP_VSA2	0.012598
18	FpDensityMorgan3	0.012595
19	VCH5	0.012523
20	maxtsC	0.012479
21	GATS4e	0.012299
22	VE3Dzs	0.012019
23	VE3Dzi	0.01185
24	fr_aniline	0.011721
25	VCH7	0.011621
26	maxaaNH	0.011566
27	RPCS	0.011367
28	AVP4	0.011362
29	RDF15m	0.01133
30	piPC10	0.011233
31	GATS6s	0.010872
32	SCH5	0.010443
33	geomShape	0.010294
34	AMR	0.010254
35	TDB2s	0.010249
36	E2m	0.010227
37	JGI3	0.010129
38	E1v	0.009976
39	ASP4	0.009928
40	TDB5r	0.009893
41	GATS5v	0.009826
42	ASP3	0.009785
43	P2i	0.009707
44	E2v	0.00952
45	AATS6i	0.009492

46	GATS5m	0.009456
47	MOMIZ	0.009286
48	maxHssNH	0.009219
49	TDB10s	0.009104
50	TDB7r	0.009001
51	SRW9	0.008998
52	fr_ketone_Topless	0.008958
53	Ti	0.00858
54	TDB8r	0.008137
55	naasC	0.007638
56	GATS8v	0.007558
57	minaaS	0.007389
58	SHBint4	0.006948
59	Ks	0.006253
60	fr_allylic_oxid	0.006151
61	P1s	0.006055
62	nFRing	0.004266
63	JGI10	0.002117
64	maxHBint10	0.000685

Appendix B: Programming code

1. Descriptor generation program

```
from rdkit import Chem
from rdkit.ML.Descriptors import MoleculeDescriptors
from rdkit.Chem import Descriptors
from padelpy import from_smiles
import re
import time

nms=[x[0] for x in Descriptors._descList]
print('\n')
calc = MoleculeDescriptors.MolecularDescriptorCalculator(nms)
f=open('/scratch/materialdesign/b3lyp_2017/datasmile_3.txt')
for i, line in enumerate(f):
    mydes=[]
    mylist=[]
    line=line.split(',')
    number=str(line[0])
    print(number,i)
    line=line[1]
    m=Chem.MolFromSmiles(line)
    try:
        time.sleep(1)
        des= from_smiles(line,fingerprints=True,timeout=180)
        des=str(des).split(',')
        for ii in range(len(des)):
            b=des[ii].split(',')
            b=des[ii].strip('[').strip(']')
            b=re.sub('[^.,a-zA-Z0-9 \n\.]', '', b)
            b=b.replace('[', ' ')
            b=b.replace(']', ' ')
            b=b.strip()
            b=b.split(' ')
            mylist.append(b[0])
        try:
```

```
        b=b[1]
    except:
        b=""
    if bool(b)==True:
        mydes.append(float(b))
    if bool(b)==False:
        mydes.append('NA')
a=calc.CalcDescriptors(m)
a=str(a)
a=a.replace('(', ')')
a=a.replace(')', ')')
line=str(line)
towrite=str(number+', '+line.strip(' ').strip('\n')+', '+a+', '+str(mydes)).strip(']')
time.sleep(3)
with open('/scratch/materialdesign/b3lyp_2017/book_dataset_3.csv', 'a') as mydata:
    mydata.write(towrite+'\n')
except:
    time.sleep(3)
```

2. EDA code (part 1)

```
#### Step 1 ####
import pandas as pd
import numpy as np
import warnings
import time

overall_start = time.time()

## initialise timing
steps = ['1.0 - Initialise import settings',
        '1.1 - First row check',
        '1.2 - Set row limit',
        '1.3 - Concat Chunks (1 file)',
        '1.4 - Remove []',
        '1.5 - Export',
        '1.1-5 - 1 file time (total)',
        'Overall']

times = {step:[] for step in steps}

#### Step 1: import, remove space before inf, NA rows, ] in last column, [ in nAcid column, ####
#### add header descriptors to each file, export as temp file #####

#### 1.0 - Initialise import settings ####
start_time = time.time() # opt

#path = 'D:/completed_code/'

## names of files to filter:

files = ['book_dataset_0',
        'book_dataset_1',
```

```
'book_dataset_2',
'book_dataset_3',
'book_dataset_4',
'book_dataset_5',
'book_dataset_6',
'book_dataset_7',
'book_dataset_8',
'book_dataset_9',
'book_dataset_10',
'book_dataset_11',
'book_dataset_12',
'book_dataset_13',
'book_dataset_14',
'book_dataset_15',
'book_dataset_16',
'book_dataset_17',
'book_dataset_18',
'book_dataset_19',
'book_dataset_20',
'book_dataset_21',
'book_dataset_22',
'book_dataset_23',
'book_dataset_24',
'book_dataset_25',
'book_dataset_26',
'book_dataset_27']
# files = ['book_dataset_0']
# files = ['book_test_14']
# files = ['book_dataset_0_modifiedsubset_7']

'''
## remove .csv first when using google drive path, also uncomment google drive path - tfiles to get
filename from path
files = ['/content/drive/MyDrive/completed sets/book_dataset_17',
        '/content/drive/MyDrive/completed sets/book_dataset_18',
        '/content/drive/MyDrive/completed sets/book_dataset_19',
```

```

'/content/drive/MyDrive/completed sets/book_dataset_27']
files = files[2:]
'''

#full_filename = [path + x for x in files]

## names of 'cleaner' version of files - (NA rows, [ ] removed): temp_filename
tfiles = ['temp_' + fname for fname in files]

'''

## google drive path - tfiles
def find_name(paths):
    return [path[path.find('book'):] for path in paths]
tfiles = ['temp_' + file for file in find_name(files)]
'''

first_file_w_descriptor = 'book_dataset_0'

# to check if all the number of columns are the same
ncol_check = []
times['1.0 - Initialise import settings'].append(time.time() - start_time)


for count, file in enumerate(files):
    file_time = time.time() # opt

    #full_filename = path + file

    ##### 1.1 - First row check #####
    start_time = time.time() # opt

```

```
# checking whether first row is descriptor
```

```
first_row = pd.read_csv('{} .csv'.format(file), nrows = 0, header = 0)
```

```
try:
```

```
    float(first_row.columns[3])
```

```
    print('Not first file, using descriptors from {} as header'.format(first_file_w_descriptor))
```

```
    header = pd.read_csv('{} .csv'.format(first_file_w_descriptor), nrows = 0, header = 0)
```

```
    descriptors = header.columns
```

```
    skiprow = None
```

```
except ValueError:
```

```
    print ("This is the first file/a file with header descriptors")
```

```
    descriptors = first_row.columns
```

```
    # skip first row (header) later since already imported headers
```

```
    skiprow = 1
```

```
times['1.1 - First row check'].append(time.time() - start_time)
```

```
#### 1.2 - Chunk ####
```

```
start_time = time.time() # opt
```

```
# number of rows to read at once
```

```
rchunk_s = 100
```

```
# setting row limit (nrows) prevents pandas to read the empty rows (after data) (happens to book-  
dataset1, doesn't happen to book_dataset_0)
```

```
row_lim = pd.read_csv('{} .csv'.format(file), usecols = [0], names = descriptors, skiprows =  
skiprow).dropna().shape[0]
```

```
# recognising " 'NA'" as NaN
```

```
navalues = "'NA'"
```

```
times['1.2 - Set row limit'].append(time.time() - start_time)
```

```
# read a file in row chunks
```



```
chunks = pd.read_csv('{} .csv'.format(file), names = descriptors, skiprows = skiprow, chunksize =
rchunk_s, nrows = row_lim, na_values = navalues, skipinitialspace = True)
```

Iteratively appending rows to a DataFrame can be more computationally intensive than a single concatenate.

A better solution is to append those rows to a list

and then concatenate the list with the original DataFrame all at once.

<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.append.html>

1.3 - Concat Chunks (1 file)

```
start_time = time.time() # opt
```

removing NA rows upon import, concat all row chunks (of a single file)

```
df = pd.concat(chunks).dropna()
```

#remove duplicates

dropping ALL duplicate values

if df.duplicated(subset=['smiles']).any():

```
    df.drop_duplicates(subset=['smiles'],inplace = True)
```

```
    print('duplicates removed')
```

else:

```
    print('no duplicates')
```

```
print('Read {} rows in {}'.format(row_lim, file))
```

```
print('NA rows removed')
```

```
print('{} rows left'.format(df.shape[0]))
```

```

times['1.3 - Concat Chunks (1 file)'].append(time.time() - start_time)

# store number of columns of file to be checked later
ncol_check.append(df.shape[1])

#### 1.4 - Remove [] ####
start_time = time.time() # opt

# removing ] in last column
if 'PubchemFP880' in df.columns:
    if type(df['PubchemFP880'].iloc[-1]) == str and "]" in df['PubchemFP880'].iloc[-1]:
        df['PubchemFP880'] = pd.to_numeric(df['PubchemFP880'].astype(str).str.rstrip(']'))
        print('] removed in last column')
    else:
        print('] not present in last column')
else:
    print('PubchemFP880 not among columns')

# removing [ in nAcid column
if 'nAcid' in df.columns:
    if type(df['nAcid'].iloc[3]) == str and "[" in df['nAcid'].iloc[0]:
        df['nAcid'] = pd.to_numeric(df['nAcid'].str.lstrip '['))
        print('[ removed in nAcid')
    else:
        print('[ not present in nAcid')
else:
    print('nAcid not among columns')
times['1.4 - Remove []'].append(time.time() - start_time)

```

```

#### 1.5 - Export ####
start_time = time.time() # opt

df.to_csv('{} .csv'.format(tfiles[count]), index=False)
print('File exported as {} .csv'.format(tfiles[count]))
print()
times['1.5 - Export'].append(time.time() - start_time)

times['1.1-5 - 1 file time (total)'].append(time.time() - file_time)

# clear variable to reduce RAM usage
chunks = None
df = None

# make sure that all files have same number of cols
if ncol_check.count(ncol_check[0]) == len(ncol_check):
    print('All files have same number ( {}) of columns'.format(ncol_check[0]))
else:
    print('****Warning: files have different numbers of columns****')
    for tfile, ncol in zip(tfiles, ncol_check):
        print ('{}: {} columns'.format(tfile, ncol))

## file naming
# take numbers only from each file
nums = []
for tfile in tfiles:
    fnums = [char for char in tfile if char.isdigit()]
    nums.append(" ".join(fnums))

```

```

if len(nums) < 5:
    nums_ = '_'.join(nums)
else: # 5 files or above
    nums_ = '_'.join(nums[:2] + ['...'] + nums[-2:])

print(tfiles)
times['Overall'].append(time.time() - overall_start)
print("--- {:.4f} seconds ---".format(time.time() - overall_start)) #opt

## timing
d = list( zip( times.keys(), [np.mean(np.array(times[key])) for key in times], times.values() ) )
time_df, time_fname = pd.DataFrame(data=d, columns = ['Step', 'Avg Time', 'Times']), 'Step 1 Times
{}'.format(nums_)
time_df.to_csv('{}.csv'.format(time_fname))
print('--- Times exported as {} ---'.format(time_fname)) # opt

'''

Not first file, using descriptors from book_dataset_0 as header
Read 149841 rows in /content/drive/MyDrive/completed sets/book_dataset_18
NA rows removed
131520 rows left
] not present in last column
[ removed in nAcid
File exported as temp_book_dataset_18.csv

Not first file, using descriptors from book_dataset_0 as header
Read 149862 rows in /content/drive/MyDrive/completed sets/book_dataset_19
NA rows removed

```

125649 rows left

] not present in last column

[removed in nAcid

File exported as temp_book_dataset_19.csv

All files have same number (2958) of columns

['temp_book_dataset_18', 'temp_book_dataset_19']

--- 1568.3261 seconds ---(overall)

--- Times exported as Step 1 Times 18_19 ---

'''

3. EDA code (part 2)

```
#### Step 2 ####

import pandas as pd
import numpy as np
import warnings
import time

overall_start = time.time()

## initialise timing
steps = ['2.0 - Initialise import settings',
        '2.1.1 - Read',
        '2.1.2 - Concat',
        '2.2 - Replace inf, uniq',
        '2.3 - Mean',
        '2.4 - Mean/median',
        '2.5 - Std/mean',
        '2.6 - Naming, exporting',
        '2.1-6 - Colchunk time',
        'Overall']
times = {step:[] for step in steps}

#### Step 2.0: initialise import settings ####
start_time = time.time() # opt

## names of files to filter:
# files = ['D:/completed_code/book_dataset_0','D:/completed_code/book_test_14']
files = ['book_dataset_0',
        'book_dataset_1',
        'book_dataset_2',
        'book_dataset_3',
        'book_dataset_4',
        'book_dataset_5',
        'book_dataset_6',
```

```

'book_dataset_7',
'book_dataset_8',
'book_dataset_9',
'book_dataset_10',
'book_dataset_11',
'book_dataset_12',
'book_dataset_13',
'book_dataset_14',
'book_dataset_15',
'book_dataset_16',
'book_dataset_17',
'book_dataset_18',
'book_dataset_19',
'book_dataset_20',
'book_dataset_21',
'book_dataset_22',
'book_dataset_23',
'book_dataset_24',
'book_dataset_25',
'book_dataset_26',
'book_dataset_27']
# files = ['book_dataset_0']
# files = ['book_test_14']
# files = ['book_dataset_17']
# files = ['book_dataset_0', 'book_test_14', 'book_dataset_0_modifiedsubset_7']
# files = ['book_dataset_0_modifiedsubset_7']

## names of 'cleaner' version of files - (NA rows, [ ] removed): temp_filename
# set files to None as they should not be used here
tfiles, files = ['temp_'+ fname for fname in files], None

## google drive path (set files to None here)
# tfiles, files = ['/content/drive/MyDrive/Research Internship/temp_book_dataset_17',
#                 '/content/drive/MyDrive/Research Internship/temp_book_dataset_18',
#                 '/content/drive/MyDrive/Research Internship/temp_book_dataset_19',

```

```

#          '/content/drive/MyDrive/Research Internship/temp_book_dataset_27'], None

# read first 2 rows to obtain headers and total number of columns
row1_2    = pd.read_csv('{}'.format(tfiles[0]), nrows = 1, header = 0)
descriptors = row1_2.columns

# identify where the numeric columns starts (ignore indices & smiles now)
start_numcol = 0
for i in range(5):
    if type(row1_2.iloc[0,i]) == str:
        start_numcol = i+1
        break

# number of numeric columns
n_numcol = len(descriptors) - start_numcol

# column chunk size to process
cchunk_s = 300

# number of column chunks (and hence files) that will be generated
n_cchunk = len(descriptors)//cchunk_s + 1

## col_ranges (to import certain column ranges of the files)

# first col chunk without numcol - size: cchunk_s - start_numcol
col_ranges = [range(start_numcol, cchunk_s)]

# most col chunks - size: cchunk_s
col_ranges.extend([range(start * cchunk_s, start * cchunk_s + cchunk_s) for start in range(1,
n_numcol//cchunk_s)])

# last col chunk - size: remainder of n_numcol/cchunk_s (< cchunk_s)

```



```

# starts at
start = (n_numcol//cchunk_s)*cchunk_s
col_ranges.append(range(start, len(descriptors)))

print('Number of numeric columns          : {} columns'.format(n_numcol))
print('Number of columns from all files processed at once: {} columns'.format(cchunk_s))
print('Number of column chunks that can be generated   : {} chunks'.format(n_cchunk))

if cchunk_s >= len(descriptors):
    raise Exception('Column chunk size should be smaller than total number of columns
({})'.format(len(descriptors)))

print('Initialisation complete\n')
times['2.0 - Initialise import settings'].append(time.time() - start_time)


## select only part of col_ranges
# to select only one chunk, let part_start = n, part_end = n+1: e.g. 5th chunk: part_start = 4, part_start
= 5
# part_start = len(col_ranges)-1 # start at last chunk
# part_start = len(col_ranges)-2 # start at 2nd last chunk
part_start = 0
#part_end = 1
part_end = len(col_ranges) # end at last chunk

print('Selected numeric column chunks to be processed   : numcol chunk {} to numcol chunk {} ({}
out of {} numcol chunks)'.format(part_start+1, part_end, part_end-part_start, n_cchunk))
print('Number of files that will be generated          : {} files'.format(part_end-part_start))

```

```

for count, col_range in enumerate(col_ranges[part_start:part_end], start = part_start + 1):
    chunk_start = time.time()

    # empty list to store each imported file
    df_files = []

    ##### Step 2.1: import selected column range for all files #####
    start_time = time.time() # opt

    for tfile in tfiles:

        # number of rows to read at once
        rchunk_s = 100

        # setting row limit (nrows) prevents pandas to read the empty rows (after data) (happens to
book-dataset1, doesn't happen to book_dataset_0)
        row_lim = pd.read_csv('{} .csv'.format(tfile), usecols = [0]).dropna().shape[0]

        # read a file in row chunks (from and up to col_range)
        chunks = pd.read_csv('{} .csv'.format(tfile), usecols = col_range, skiprows = 1, chunksize =
rchunk_s, nrows = row_lim, names = descriptors[col_range[0]:col_range[-1]+1], keep_default_na
= False)

        # add the file to df_files
        df_files.append(pd.concat(chunks))

    print('read through all files\n')
    times['2.1.1 - Read'].append(time.time() - start_time)

    start_time = time.time() # opt

    df_num = pd.concat(df_files)
    # clear variables to reduce RAM usage
    chunks = None
    df_files = None

```

```

print('numcol chunk {} created'.format(count))
print('{} rows'.format(df_num.shape[0]))
print('{} columns left\n'.format(df_num.shape[1])) # is this left chunks??
times['2.1.2 - Concat'].append(time.time() - start_time)

#### Step 2.2: replacing inf with column max value, remove columns with few unique values ####
start_time = time.time() # opt

df_num = df_num.apply(pd.to_numeric, errors = 'raise')

# selcting only the columns that contain inf
# assuming that there are no NaN values right after importing (since all NaN rows are removed),
# so after replacing all infs with NaN, all NaNs are inf

# temporarily replaces infs with NaN
infsreplaced = df_num.mask(df_num == np.inf)
# columns without inf
noinf = infsreplaced.dropna(axis = 1)
# columns with inf
infs = df_num.drop(noinf.columns, axis = 1)

cols = infs.columns
## replacing infs with col max value (by first excluding inf)
for col in cols:
    # create a boolean Series (the mask) that has all inf as False
    mask = infs[col] != np.inf
    # setting inf to max (False values (inf) are converted to max in column (inf ignored by mask))
    df_num.loc[~mask, col] = df_num.loc[mask, col].max()
print("inf replaced with col max value")

# clear variables to reduce RAM usage
infsreplaced = None

```

```

noinf      = None
infs       = None

# number of unique values in a column
n_uniq_values = np.array(df_num.apply(lambda x: len(x.unique()))))
## remove columns that have (2) unique values and below
uniq_thres = 2
df_num = df_num.loc[:, n_uniq_values > uniq_thres]

print('cols with {} unique values and below removed'.format(uniq_thres))
print('{} columns left\n'.format(df_num.shape[1]))

# around col 1800-2100 , uniq values starts to be <2
# last file might not be generated in this case if whole numcol chunk falls after this range
if df_num.shape[1] == 0:
    print('no column left, file will not be generated')
    print('--- last file already exported as Filtered {}-{} --- \n\n\n\n\n'.format(nums_, fi))
    continue

times['2.2 - Replace inf, uniq'].append(time.time() - start_time)

##### Step 2.3: calculate mean, remove columns with 0 mean, dealing with inf mean #####
start_time = time.time() # opt

## mean

# splitting into n minor column chunks
n = 10
split = np.array_split(np.array(df_num), n, axis = 1)

# first calculation of mean will result in overflow error (ignored here)
# caused by a few columns with numbers that are too large,

```

```

# which result in inf mean (doesn't matter if median within assumption)
with warnings.catch_warnings():
    warnings.simplefilter("ignore")
    # nanmean ignores NaN if present
    mean = [m for colchunk in split for m in np.nanmean(colchunk, axis = 0)]

print('column mean calculated\n')
times['2.3 - Mean'].append(time.time() - start_time)

##### Step 2.4: remove columns with mean/median > mm_thres (300), or columns where
median==0 but mean > mean_if_md0_thres (300) #####
start_time = time.time() # opt

# mm stands for mean/median
mm_thres = 300
mean_if_md0_thres = 300

mean = np.array(mean)
median = np.array(df_num.median())

# assume that all median < max_calc_mean/mm_thres(300), returns warning otherwise
nrows = df_num.shape[0] # should be around 4,050,000 (27 files * 150,000 rows)
max_stor_val = np.finfo(1.0).max
max_calc_mean = max_stor_val/nrows
assump_med = max_calc_mean/mm_thres
out_med_colname = df_num.columns[median > assump_med]

if len(out_med_colname)>0:
    print('***Warning: some median larger than assumed, outliers are {}, check
mean/median again for inf mean by scaling.'.format(out_med_colname))
else:
    print('all median within assumption, safe to reject mean/median == np.inf')

```

```

# ignore errors caused by dividing by median = 0 (returns inf)
with np.errstate(divide='ignore', invalid = 'raise'):
    mm = mean/median

## mean/median < threshold - accept
## mean != 0, median = 0 - accept only if mean < 300
print('{} columns left'.format(df_num.shape[1]))

df_num.loc['mean'] = mean
df_num = df_num.loc[:, np.array(mm<mm_thres) | (np.array(median==0) &
np.array(mean<mean_if_md0_thres)) ]

mean = np.array(df_num.loc['mean'].values)
df_num.drop(index = 'mean', inplace = True)

print('cols with mean/median > {} removed; if median = 0, cols with mean > {}
removed'.format(mm_thres, mean_if_md0_thres))
print('{} columns left\n'.format(df_num.shape[1]))
times['2.4 - Mean/median'].append(time.time() - start_time)


#### Step 2.5: removing columns that don't satisfy std/mean condition ####
start_time = time.time() # opt

# calculate std
# splitting into n column chunks
n = 10
split = np.array_split(np.array(df_num), n, axis = 1)

std = np.array([s for colchunk in split for s in np.nanstd(colchunk, axis = 0)])

```

```

## std/mean condition: std/mean > 0.1 - accept
stdmean_thres = 0.1

stdmean = std/mean

print('{} columns left'.format(df_num.shape[1]))

df_num = df_num.loc[:, abs(stdmean) > stdmean_thres]

print('cols with |std/mean| < {} removed'.format(stdmean_thres))
print('{} columns left\n'.format(df_num.shape[1]))
times['2.5 - Std/mean'].append(time.time() - start_time)


#### Step 2.6: file naming, exporting ####
start_time = time.time() # opt

## file naming
# take numbers only from each file
nums = []
for tfile in tfiles:
    fnums = [char for char in tfile if char.isdigit()]
    nums.append('.'.join(fnums))

if len(nums) < 5:
    nums_ = '_'.join(nums)
else: # 5 files or above
    nums_ = '_'.join(nums[:2] + ['...'] + nums[-2:])

# file index (1 to n_cchunk)
if count == 1:
    fi = str(count)+' (first)'

## concat only the FIRST numeric col chunk with smiles (and index)

```

```

# empty list to store each imported smile
df_smiles = []

for tfile in tfiles:

    # number of rows to read at once
    rchunk_s = 100

    # setting row limit (nrows) prevents pandas to read the empty rows (after data) (happens to
book-dataset1, doesn't happen to book_dataset_0)
    row_lim = pd.read_csv('{} .csv'.format(tfile), usecols = [0]).dropna().shape[0]

    # read a file in row chunks (up to smiles column)
    chunks = pd.read_csv('{} .csv'.format(tfile), usecols = range(start_numcol), skiprows = 1,
chunksize = rchunk_s, nrows = row_lim, names = descriptors[:start_numcol], keep_default_na =
False)

    # add the file to df_smiles
    df_smiles.append(pd.concat(chunks))

df_smiles = pd.concat(df_smiles)

# joining back the indices and smile columns
df_num = pd.concat([df_smiles, df_num], axis = 1)
print('rejoined with the indices and smile columns')
print('{} columns left'.format(df_num.shape[1]))

# clear variables to reduce RAM usage
df_smiles = None
chunks = None

elif count == n_cchunk:
    fi = str(count)+' (last)'
else: # count == 2, 3, ... n_cchunk-1
    fi = str(count)

# export

```



```

df_num.to_csv('Filtered {}-{}.csv'.format(nums_, fi), index=False)
# display(df_num)

# clear variable to reduce RAM usage
df_num = None

print('--- File exported as Filtered {}-{} ----\n'.format(nums_, fi)) # opt
print('Numcol chunk {} completed. \n\n\n\n\n'.format(fi))
times['2.6 - Naming, exporting'].append(time.time() - start_time)
times['2.1-6 - Colchunk time'].append(time.time() - chunk_start)

times['Overall'].append(time.time() - overall_start)
print("--- {:.4f} seconds ---(overall)".format(time.time() - overall_start)) #opt


## timing
d = list( zip( times.keys(), [np.mean(np.array(times[key])) for key in times], times.values() ) )
time_df, time_fname = pd.DataFrame(data=d, columns = ['Step', 'Avg Time', 'Times']), 'Step 2 Times
{} ({} chunks)'.format(nums_, n_cchunk)
time_df.to_csv('{}{}.csv'.format(time_fname))
print('--- Times exported as {} ---'.format(time_fname)) # opt


'''
Number of numeric columns          : 2956 columns
Number of columns from all files processed at once: 300 columns
Number of column chunks that can be generated    : 10 chunks
Initialisation complete

Selected numeric column chunks to be processed   : numcol chunk 4 to numcol chunk 5 (2 out of 10
numcol chunks)
Number of files that will be generated           : 2 files

```

read through all files

numcol chunk 4 created

403741 rows

300 columns left

inf replaced with col max value

cols with 2 unique values and below removed

163 columns left

column mean calculated

all median within assumption, safe to reject mean/median == np.inf

163 columns left

cols with mean/median > 300 removed; if median = 0, cols with mean > 300 removed

155 columns left

155 columns left

cols with |std/mean| < 0.1 removed

155 columns left

--- File exported as Filtered 17_18_19_27-4 ----

Numcol chunk 4 completed.

read through all files

numcol chunk 5 created

403741 rows

300 columns left

inf replaced with col max value

cols with 2 unique values and below removed

218 columns left

column mean calculated

all median within assumption, safe to reject mean/median == np.inf

218 columns left

cols with mean/median > 300 removed; if median = 0, cols with mean > 300 removed

205 columns left

205 columns left

cols with |std/mean| < 0.1 removed

186 columns left

--- File exported as Filtered 17_18_19_27-5 ----

Numcol chunk 5 completed.

--- 999.6358 seconds ---(overall)

--- Times exported as Step 2 Times 17_18_19_27 (10 chunks) ---

'''

4. EDA code (part 3)

```
import pandas as pd

# files = ['D:/completed_code/smiles/Filtered 0-1 (first).csv',
#          'D:/completed_code/smiles/Filtered 0-2.csv',
#          'D:/completed_code/smiles/Filtered 0-3.csv']

df1= pd.read_csv('Filtered 0_1_..._26_27-1 (first).csv')
df2= pd.read_csv('Filtered 0_1_..._26_27-2.csv')
df3= pd.read_csv('Filtered 0_1_..._26_27-3.csv')
df4= pd.read_csv('Filtered 0_1_..._26_27-4.csv')
df5= pd.read_csv('Filtered 0_1_..._26_27-5.csv')
df6= pd.read_csv('Filtered 0_1_..._26_27-6.csv')
df7= pd.read_csv('Filtered 0_1_..._26_27-7.csv')

df_final = pd.concat([df1,df2,df3,df4,df5,df6,df7], axis = 1)

#dat1 = pd.concat([dat1, dat2], axis=1)

df_final.to_csv('Final_compiled_dataset.csv')

print(df_final.shape)
```

5. DFT calculation extraction code

```
# https://stackoverflow.com/questions/2018512/reading-tar-file-contents-without-untarring-it-in-python-script
# solve value length error # https://stackoverflow.com/questions/2018512/reading-tar-file-contents-without-untarring-it-in-python-script

import os
import tarfile
import lzma
import cclib
import pandas as pd
import time

start_time = time.time()

file_num = 1

directory1 = '/lustre/project/materialdesign/b3lyp_2017/complete_tar/b3lyp_2017'
#directory1 = 'D:/completed_code'

os.chdir('{}{}'.format(directory1, file_num))
print(os.getcwd())

# before extract, create dataframe
id_name = []
HOMO = []
LUMO = []
first_excited_state = []
osci_strength = []

id_removed = []

for filename in os.listdir():
    print('Current compound:',filename)
    print("-----")
```

```

try:
    tar_1 = tarfile.open(filename)
    tar_1.extractall()

    tar_1.close()

    os.remove(filename) # remove the extracted file

    os.chdir('{}/{}/{}'.format(directory1, file_num, filename.replace('.tar.gz','')))
    print("Going into ", os.getcwd())

except:
    os.chdir('{}/{}/{}'.format(directory1, file_num, filename.replace('.tar.gz','')))
    print("Going into ", os.getcwd())

# In the compound file

for files in os.listdir():

    directory = '{}/{}/{}'.format(directory1, file_num, filename.replace('.tar.gz',''), files)
    if os.path.isdir(directory):
        os.chdir(directory)
    else:
        os.remove(directory)

    print('Current number: ', files)
    #id.append(files)
    print("Current file: \n",os.getcwd())

    print('Number of files: ',len(os.listdir()))
    print("The files inside: \n",os.listdir())
    print("\n")

    if len(os.listdir()) < 5:
        id_removed.append(files)

```

```

# find and untar +d log xz
for logfile in os.listdir():
    if logfile.endswith('.td-b3lyp_6-31g+(d).log.xz'):
        print('Found: ',logfile)

    try:
        with lzma.open(logfile) as f, open('{}'.format(logfile.replace('.xz','')), 'wb') as fout:
            file_content = f.read()
            fout.write(file_content)
        print("logfile extracted")
    except:
        print('logfile NA')

# extract homo lumo

for extracted_log in os.listdir():
    if extracted_log.endswith('.log'):
        try:
            print("Extracting, found log: ", extracted_log)
            id_name.append("{}".format(extracted_log.replace('.td-b3lyp_6-31g+(d).log','')))

            try:
                data = cclib.io.ccread(extracted_log)

                try:
                    homo=data.moenergies[0][int(data.homos)]
                    print("HOMO: ",homo)
                    HOMO.append(homo)
                except:
                    print("Error getting HOMO")
                    HOMO.append('NA')
            except:
                pass
        except:
            pass

```

```
try:
    lumo=data.moenergies[0][int(data.homos)+1]
    print("LUMO: ",lumo)
    LUMO.append(lumo)
except:
    print("Error getting LUMO")
    LUMO.append('NA')
```

```
try:
    first_excitedstate=0.000123981*data.etenergies[0]
    print("First_exc: ",first_excitedstate)
    first_excited_state.append(first_excitedstate)
except:
    print("Error getting first excited state")
    first_excited_state.append('NA')
```

```
try:
    oscillator_strength=data.etoscs[0]
    print("Osi_st: ",oscillator_strength)
    osci_strength.append(oscillator_strength)
except:
    print("Error getting oscillator strength")
    osci_strength.append('NA')
```

```
except:
    print("Error cclib extract file")
    id_name.remove("{}".format(extracted_log.replace('.td-b3lyp_6-31g+(d).log','')))
```

```
except:
    print('log not found')
```

```
print("\n")
```



```
print("#####")
# Back to main file
os.chdir('{}{}'.format(directory1, file_num))
print("Back to ",os.getcwd())
print()

data = pd.DataFrame(
    {
        'id':id_name,
        'HOMO':HOMO,
        'LUMO':LUMO,
        'first_excited_state': first_excited_state,
        'oscillator_strength': osci_strength
    }
)

print(data)
print('id_removed: ',id_removed)

data.to_csv('{}extracted_data_{}.csv'.format(directory1, file_num),index=False)

end_time = time.time()
hours, rem = divmod(end_time - start_time, 3600)
minutes, seconds = divmod(rem, 60)
print("Time elapsed: {:0>2}:{:0>2}:{:05.2f}".format(int(hours), int(minutes), seconds))
```

6. Statistics of the electronic properties

```
# combine together

import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

sns.set(rc={'figure.figsize':(20, 20)})

sns.set(style = 'darkgrid')

# directory1 = 'D:/completed_code/randomf/trial'
# directory2 = 'D:/completed_code/randomf/trial'
directory1 = '/lustre/project/materialdesign/b3lyp_2017/Finalist'
directory2 = '/lustre/project/materialdesign/b3lyp_2017/Finalist/stats'
# Final_combined_inner.csv
df = pd.read_csv('{} /Final_combined_inner.csv'.format(directory1))

# step 2

range_HOMO1 = df[(df['HOMO'] < -8)].index
range_HOMO2 = df[(df['HOMO'] > -3)].index
df.drop(range_HOMO1, inplace = True )
df.drop(range_HOMO2, inplace = True)

range_LUMO1 = df[(df['LUMO'] < -4)].index
range_LUMO2 = df[(df['LUMO'] > 0)].index
df.drop(range_LUMO1, inplace = True)
df.drop(range_LUMO2, inplace = True)

value_osci_0 = df[(df['oscillator_strength'] == 0)].index
df.drop(value_osci_0, inplace = True)
```

```

df['log_oscillator_strength'] = np.log10(df['oscillator_strength'])

print('-----')
print('shape after:\n',df.shape)

ax5 = sns.displot(df, x = 'HOMO', kind = 'hist', aspect = 1.2, )
plt.xlabel('DFT calculated HOMO (eV)', fontsize = 'large')
plt.ylabel('Frequency', fontsize = 'large')
plt.savefig('{} /HOMO_dist_2.png'.format(directory2))

ax6 = sns.displot(df, x = 'LUMO', kind = 'hist', aspect = 1.2)
plt.xlabel('DFT calculated LUMO (eV)', fontsize = 'large')
plt.ylabel('Frequency', fontsize = 'large')
plt.savefig('{} /LUMO_dist_2.png'.format(directory2))

ax7 = sns.displot(df, x = 'log_oscillator_strength', kind = 'hist', aspect = 1.2)
plt.xlabel('DFT calculated oscillator strength (log10)', fontsize = 'large')
plt.ylabel('Frequency', fontsize = 'large')
plt.savefig('{} /osci_dist_2.png'.format(directory2))

ax8 = sns.displot(df, x = 'first_excited_state', kind = 'hist', aspect = 1.2)
plt.xlabel('DFT calculated first excited state (eV)', fontsize = 'large')
plt.ylabel('Frequency', fontsize = 'large')
plt.savefig('{} /firstex_dist_2.png'.format(directory2))

print('\nThe statistics of the distribution after:\n')
print('HOMO\n',df['HOMO'].describe(),'\n')
print('LUMO\n',df['LUMO'].describe(),'\n')
print('first_excited_state\n',df['first_excited_state'].describe(),'\n')
print('oscillator_strength\n',df['oscillator_strength'].describe(),'\n')
print('log_oscillator_strength\n',df['log_oscillator_strength'].describe(),'\n')

```

```
# step 3
```

```
df.to_csv('{}Final_combined_inner_removedHOMOLUMOlog.csv'.format(directory1), index =  
False)
```

7. Random Forest code

```
# final final

# (put in HPC) corr + bpruta+ randomserchcv success  retry + output buffer + optimize boruta

# https://blog.exploratory.io/finding-variable-importance-with-random-forest-boruta-28badd116197
# https://towardsdatascience.com/simple-example-using-boruta-feature-selection-in-python-8b96925d5d7a

# first_excited_state

import pandas as pd
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
import numpy as np

from sklearn.metrics import mean_absolute_error, mean_squared_error
#https://youtu.be/L5sxEr77Yvo?t=252
from matplotlib.pyplot import figure

import matplotlib.pyplot as plt
#from itertools import product

import time

from sklearn.feature_selection import SelectFromModel
from sklearn.feature_selection import RFECV

from boruta import BorutaPy
from pprint import pprint
from sklearn.model_selection import RandomizedSearchCV

from sklearn.metrics import r2_score

# import pydot
```

```
# from sklearn.tree import export_graphviz

import sys

class Unbuffered(object):
    def __init__(self, stream):
        self.stream = stream
    def write(self, data):
        self.stream.write(data)
        self.stream.flush()
    def writelines(self, datas):
        self.stream.writelines(datas)
        self.stream.flush()
    def __getattr__(self, attr):
        return getattr(self.stream, attr)

sys.stdout = Unbuffered(sys.stdout)

start_time = time.time()

# ----- load dataset -----

directory1 = '/lustre/project/materialdesign/b3lyp_2017/Finalist'
# directory1 = 'D:/completed_code/randomf/combine all corr boruta test'

# target = 'HOMO'
# target = 'LUMO'
target = 'first_excited_state'
# target = 'oscillator_strength'

enter_threshold = 0.90 # !!!!!!!!!!!!!!!!!!!!!!!
sample_frac = 0.005 # !!!!!!!!!!!!!!!!!!!!!!! 0.005 for dataset
boruta_max_iter = 100 # !!!!!!!!!!!!!!! 100 or 50
```

```

print("Preparing Random forest for target {}".format(target))
# Load dataset
# Final_combined_inner_rand2000_2_removedHOMOLUMOlog.csv
df = pd.read_csv('{} /Final_combined_inner_removedHOMOLUMOlog.csv'.format(directory1))

df.drop(['Unnamed: 0', 'smiles', 'HOMO', 'LUMO', 'oscillator_strength',
'log_oscillator_strength'], axis = 1, inplace = True) #!!!!!!!!!!!!!!

# here remove smiles and index first
print('dataframe shape: ', df.shape)
X = df.drop(target, axis = 1)
#print("X:\n",X) #left only fingerprint

y = df['{}'.format(target)] #!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
#print("Y:\n", y)

print("X shape: ",X.shape)
print("Y shape: ",y.shape)

# ----- load dataset complete -----

# -----correlation with target part-----
#      https://datascience.stackexchange.com/questions/39137/how-can-i-check-the-correlation-between-features-and-target-variable

correlation = df[df.columns[1:]].corr()['{}'.format(target)][:]

correlation.to_csv('{} /correlation with target {}.csv'.format(directory1, target))

print()
print('Correlation with target saved to csv')
print('Correlation of features (descriptors) with target {}'.format(target))
print(correlation)

# -----end correlation with target part-----

```

```

# -----drop correlaton -----

print()
print('Preparing to drop correlated features(descriptors)')

#   https://towardsdatascience.com/how-to-use-pairwise-correlation-for-robust-feature-selection-20a60ef7d10

def identify_correlated(df, threshold):
    """
    A function to identify highly correlated features.
    """
    # Compute correlation matrix with absolute values
    matrix = df.corr().abs()

    # Create a boolean mask
    mask = np.triu(np.ones_like(matrix, dtype=bool))

    # Subset the matrix
    reduced_matrix = matrix.mask(mask)

    reduced_matrix.to_csv('{}reduced_matrix_correlation {}.csv'.format(directory1, target))
    print('Descriptor correlation saved to csv')

    # Find cols that meet the threshold
    to_drop = [c for c in reduced_matrix.columns if \
        any(reduced_matrix[c] > threshold)]

    return to_drop

to_drop = identify_correlated(X, threshold = enter_threshold)

```



```
print()
print('{} features descriptors will be dropped:'.format(len(to_drop)))

print(to_drop)

X = X.drop(to_drop, axis = 1)

print()
print('X shape after correlation removed: ',X.shape)

#----- drop correlation end-----

##-----initialize Boruta-----

# https://www.kaggle.com/rsmits/feature-selection-with-boruta

print()
print('Performing boruta:')

print('Start sampling form the full dataset:')

df = df.iloc[0:0]

print('Concat X and y')
df = pd.concat([X, y], axis = 1)

print('Current df shape: ', df.shape)

df_sample = df.sample(frac = sample_frac, random_state = 42)

print()
print('df_sample shape: ', df_sample.shape)
```

```
# clear original df
df = df.iloc[0:0]

X_sample = df_sample.drop(target, axis = 1)
#print("X:\n",X) #left only fingerprint

y_sample = df_sample['{}'.format(target)]

print()
print('Iterating boruta:')

model = RandomForestRegressor(n_estimators = 100, max_depth = 5, random_state = 42, n_jobs =
-1) # set random_state

# fit random forest model to the training data
# model.fit(X, y)


# define Boruta feature selection method
feat_selector = BorutaPy(model, n_estimators='auto', verbose=2, random_state=42, max_iter =
boruta_max_iter , perc = 90)


# find all relevant features
feat_selector.fit(np.array(X_sample), np.array(y_sample))

# check selected features
feat_selector.support_

# check ranking of features
feat_selector.ranking_

feat_selector.n_features_
```

```

print()
print('Boruta model currently in use:')
pprint(featur_selector.get_params())
print()

### print results
green_area = X_sample.columns[featur_selector.support_].to_list()
blue_area = X_sample.columns[featur_selector.support_weak_].to_list()

print('features in the green area:', green_area)
print('features in the blue area:', blue_area)

# zip column names, ranks, and decisions in a single iterable
feature_ranks = list(zip(X_sample.columns,
                        featur_selector.ranking_,
                        featur_selector.support_))

# iterate through and print out the results
for feat in feature_ranks:
    print('Feature: {:<25} Rank: {}, Keep: {}'.format(feat[0], feat[1], feat[2]))

# call transform() on X to filter it down to selected features
# X = featur_selector.transform(np.array(X))

selected_features = []
selected_features.extend(green_area)
selected_features.extend(blue_area)
selected_features.append('{}' .format(target))
print('selected_features:',selected_features)

print()
print('Number of selected features:',len(selected_features))
print()

# -----boruta end -----

```

```

# ----- reload new dataframe-----

df = pd.read_csv('{}Final_combined_inner_removedHOMOLUMOlog.csv'.format(directory1),
usecols= selected_features)

print('df shape after boruta',df.shape)

X = df.drop(target, axis = 1)
#print("X:\n",X) #left only fingerprint

y = df['{}'.format(target)]
#print("Y:\n", y)

print("X shape: ",X.shape)
print("Y shape: ",y.shape)

# -----reload new dataframe end-----

# ----- finding best parameters -----

# print('Finding best parameters: ')

# X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state = 42)
# print("X_train shape: ",X_train.shape)
# print("y_train shape: ",y_train.shape)

# print("X_test shape: ",X_test.shape)
# print("y_test shape: ",y_test.shape)

# -----

# # Number of trees in random forest

```

```

# n_estimators = [int(x) for x in np.linspace(start = 200, stop = 2000, num = 10)]

# # Number of features to consider at every split
# max_features = ['auto', 'sqrt', 'log2']

# criterion = ['squared_error', 'absolute_error', 'poisson' ]

# # Maximum number of levels in tree
# max_depth = [int(x) for x in np.linspace(10, 110, num = 11)]
# max_depth.append(None)

# # Minimum number of samples required to split a node
# min_samples_split = [2, 5, 10]

# # Minimum number of samples required at each leaf node
# min_samples_leaf = [1, 2, 4]

# # Method of selecting samples for training each tree
# # bootstrap = [True, False]

# # Create the random grid
# random_grid = {'n_estimators': n_estimators,
#                 # 'max_features': max_features,
#                 # 'max_depth': max_depth,
#                 # 'min_samples_split': min_samples_split,
#                 # 'min_samples_leaf': min_samples_leaf,
#                 # # 'criterion': criterion,
#                 # # 'bootstrap': bootstrap
#                 }
# pprint(random_grid)

# # Use the random grid to search for best hyperparameters
# # First create the base model to tune
# rf = RandomForestRegressor(random_state = 42)

```

```
# # Random search of parameters, using 5 fold cross validation,
# # search across 50 different combinations, and use all available cores
# search = RandomizedSearchCV(estimator = rf,
#                             param_distributions = random_grid,
#                             n_iter = 50,
#                             cv = 5,
#                             verbose=1,
#                             random_state = 42,
#                             n_jobs = -1)

# # -----
# # Fit the random search model
# search_model = search.fit(X_train, y_train)

# dict = search_model.best_params_

# best_n_estimators = dict.get('n_estimators')
# best_max_depth = dict.get('max_depth')
# best_max_features = dict.get('max_features')
# best_min_samples_split = dict.get('min_samples_split')
# best_min_samples_leaf = dict.get('min_samples_leaf')
# best_criterion = dict.get('criterion')

# print("\nAfter performing randomized search cv: ")
# print("The best n_estimators:", best_n_estimators)
# print("The best max_depth:", best_max_depth)
# print("The best max features:", best_max_features)
# print("The best min_samples_split:", best_min_samples_split)
# print("The best min_samples_leaf:", best_min_samples_leaf)
# print("The best criterion:", best_criterion)
```

```

# print('Best estimator: ', search_model.best_estimator_)
# print('best parameter: ', dict)
# print('accuracy score: ', search_model.best_score_)
# print('model results: ', search_model.cv_results_)

# ----- find best parameter end -----

# ----- start split and random forest -----

print()
print('Start split and perform random forest:')

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state = 42)
print("X_train shape: ",X_train.shape)
print("Y_train shape: ",y_train.shape)

print("X_test shape: ",X_test.shape)
print("Y_test shape: ",y_test.shape)

# https://youtu.be/aXw12xFVxwg?t=283

# model = RandomForestRegressor(n_estimators = best_n_estimators,
#                               max_depth = best_max_depth,
#                               max_features = best_max_features,
#                               min_samples_split = best_min_samples_split,
#                               min_samples_leaf = best_min_samples_leaf,
#                               random_state = 42,
#                               oob_score = True,
#                               #criterion = best_criterion,
#                               n_jobs = -1) # set random_state

# ----- start split and random forest -----

```

```
model = RandomForestRegressor(n_estimators = 1100,  
                             random_state = 42,  
                             oob_score = True,  
                             verbose = 2,  
                             n_jobs = -1) # set random_state
```

```
print()
```

```
# Look at parameters used by our current forest
```

```
print('RFR Parameters currently in use:\n')
```

```
pprint(model.get_params())
```

```
print()
```

```
# fit random forest model to the training data
```

```
model.fit(X_train, y_train)
```

```
r1 = model.score(X_train, y_train)
```

```
print("The training accuracy is: ",r1)
```

```
r2 = model.score(X_test, y_test)
```

```
print("The testing accuracy is: ",r2)
```

```
# do prediction using test data
```

```
y_pred = model.predict(X_test)
```

```
# calculating performance
```

```
print('oob score:', model.oob_score_)
```

```
m_abs_err = mean_absolute_error(y_test, y_pred)
```

```
m_sqr_err = mean_squared_error(y_test, y_pred)
```

```
print("Mean absolute error of test model: ", m_abs_err)
```

```
print("Mean squared error of test model: ", m_sqr_err)
```



```

# https://scikit-learn.org/stable/modules/generated/sklearn.metrics.r2_score.html
r3 = r2_score(y_test, y_pred)
print("The y_test y_pred r2 score is :", r3)

print()
y_train_pred = model.predict(X_train)

m_abs_err2 = mean_absolute_error(y_train, y_train_pred)
m_sqr_err2 = mean_squared_error(y_train, y_train_pred)

print("Mean absolute error of train model: ", m_abs_err2)
print("Mean squared error of train model: ", m_sqr_err2)

# ----- inspect the predicted value -----
print()
print('Inspecting predicted value:')
df_inspect = pd.DataFrame({'Real values y_test':y_test, 'Predicted values y_pred':y_pred})
print(df_inspect)

df_inspect.to_csv('{}real values and predicted value {}.csv'.format(directory1, target))

# -----inspect predicted value end-----

# make scatter plot of experimental vs predicted target values
sns.set(color_codes = True)
sns.set_style("white")

# oob error: not suitable for large dataset

```

```

ax = sns.regplot(x = y_test, y = y_pred, scatter_kws = {'alpha': 0.8, 's' : 1, "color": "blue"},
line_kws={'color': "red", 'linewidth':0.5})
ax.set_xlabel('DFT calculated first excited state (eV)', fontsize = 20, fontweight = 'bold')
ax.set_ylabel('ML predicted first excited state (eV)', fontsize = 20, fontweight = 'bold')
plt.xticks(fontsize=20)
plt.yticks(fontsize=20)
# ax.set_xlim(0, 12)
# ax.set_ylim(0, 12)
ax.figure.set_size_inches(22, 20)
#plt.show()
plt.savefig('{} / {} / Scatter_Plot_Exp_Vs_target_{}.png'.format(directory1, target, target))

# -----end split train rf -----

# -----full importance-----

# https://youtu.be/ChWP8kzE74?list=PLpoCVQU4m6j-BRSdhXHKOwh7yTPqKUD7&t=186
importance = model.feature_importances_
columns = X_train.columns

Graph = pd.Series(importance, columns)

Graph.sort_values(ascending=False, inplace = True)
Graph.to_csv('{} / full_importance for target {}.csv'.format(directory1, target))

print("\n")
print("Descriptor importance in target {}".format(target))
print(Graph.head(20))
print("\n")
print(Graph.tail(20))

figure(figsize = (150, 100))
Graph.sort_values().plot.barh(color = "red")

```

```

plt.title("Random forest regressor model importance visualization for target {}".format(target),
fontsize = 80)
plt.xticks(fontsize=20)
plt.yticks(fontsize=20)
plt.xlabel('Descriptors Importance Score', fontsize=80)
plt.ylabel('Descriptors', fontsize=80)
plt.grid(True)
plt.savefig('{}{}/descriptor importance {}.png'.format(directory1, target, target))

#-----importance part -----

#
https://chrisalbon.com/code/machine\_learning/trees\_and\_forests/feature\_selection\_using\_random\_f
orest/
print()

# for feature in zip(X, model.feature_importances_):
#     print(feature)

# Create a selector object that will use the random forest classifier to identify
# features that have an importance of more than threshold

# sfm = SelectFromModel(model, threshold = 0.01)

# # Train the selector
# sfm.fit(X_train, y_train)

# # Print the names of the most important features
print()
print('-----most important features-----')
print()

# length = 0

```

```

# for feature_list_index in sfm.get_support(indices = True):
#     print(X.columns[feature_list_index])
#     length += 1

# print('Number of important features greater than threshold: ', length)

figure(figsize = (30, 20))
Graph2 = Graph.sort_values(ascending=False).head(20)
Graph2.sort_values().plot.barh(color = 'red')
plt.title("Random forest regressor model top 20 important descriptors for target {}".format(target),
fontsize = 25)
plt.xticks(fontsize=20)
plt.yticks(fontsize=20)
plt.xlabel('Descriptors Importance Score', fontsize=20)
plt.ylabel('Descriptors', fontsize=20)
plt.grid(True)
plt.savefig('{} / {} / top 20 important descriptors {}.png'.format(directory1, target, target))

#-----

# ----- graphviz -----

# Use sklearn to export the tree

# Write the decision tree as a dot file (success, but too large)
# visual_tree = model.estimators_[12]
# export_graphviz(visual_tree, out_file = '{} / best_tree.dot'.format(directory1), feature_names =
X.columns,
#         precision = 2, filled = True, rounded = True, max_depth = None)

# # Use pydot for converting to an image file

# # Import the dot file to a graph and then convert to a png

```

```
# (graph, ) = pydot.graph_from_dot_file('{} /best_tree.dot'.format(directory1))
# graph.write_png('{} /best_tree.png'.format(directory1))

# -----graphviz end -----

end_time = time.time()
hours, rem = divmod(end_time - start_time, 3600)
minutes, seconds = divmod(rem, 60)
print("Time elapsed: {:0>2}:{:0>2}:{:05.2f}".format(int(hours), int(minutes), seconds))
```

Appendix C: Hyperparameter of Boruta used in the research

```
'alpha': 0.05,  
'estimator': RandomForestRegressor(max_depth=5, n_estimators=503, n_jobs=-1,  
                                   random_state=RandomState(MT19937) at 0x7F8BADBB5340),  
'estimator__bootstrap': True,  
'estimator__ccp_alpha': 0.0,  
'estimator__criterion': 'squared_error',  
'estimator__max_depth': 5,  
'estimator__max_features': 'auto',  
'estimator__max_leaf_nodes': None,  
'estimator__max_samples': None,  
'estimator__min_impurity_decrease': 0.0,  
'estimator__min_samples_leaf': 1,  
'estimator__min_samples_split': 2,  
'estimator__min_weight_fraction_leaf': 0.0,  
'estimator__n_estimators': 503,  
'estimator__n_jobs': -1,  
'estimator__oob_score': False,  
'estimator__random_state': RandomState(MT19937) at 0x7F8BADBB5340,  
'estimator__verbose': 0,  
'estimator__warm_start': False,  
'max_iter': 100,  
'n_estimators': 'auto',  
'perc': 90,  
'random_state': RandomState(MT19937) at 0x7F8BADBB5340,  
'two_step': True,  
'verbose': 2
```

Appendix D: Hyperparameter of random forest used in the research

```
'bootstrap': True,  
'ccp_alpha': 0.0,  
'criterion': 'squared_error',  
'max_depth': None,  
'max_features': 'auto',  
'max_leaf_nodes': None,  
'max_samples': None,  
'min_impurity_decrease': 0.0,  
'min_samples_leaf': 1,  
'min_samples_split': 2,  
'min_weight_fraction_leaf': 0.0,  
'n_estimators': 1100,  
'n_jobs': -1,  
'oob_score': True,  
'random_state': 42,  
'verbose': 2,  
'warm_start': False
```